HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA ĐA PHƯƠNG TIỆN

BÁO CÁO BÀI TẬP LỚN

XÂY DỰNG HỆ THỐNG QUẢN LÝ THIẾT BỊ IOT CHO VƯỜN THÔNG MINH

Giảng viên môn học : TS. Nguyễn Quốc Uy

Môn học : Phát triển ứng dụng IoT

Sinh viên : Phạm Hà Thắng

Mã sinh viên : B22DCPT261

Hà Nội, 2026

LỜI CẢM ƠN

Lời đầu tiên em xin chân thành cảm ơn các thầy cô trong khoa Công nghệ Đa phương tiện của Học viện Công nghệ Bưu Chính Viễn Thông, những người đã trực tiếp giảng dạy cung cấp kiến thức và phương pháp trong thời gian qua, đó là những nền tảng cơ bản, là những hành trang vô cùng quý giá để em có thể bước vào sự nghiệp trong tương lai.

Để có được kết quả này, em xin đặc biệt gửi lời cảm ơn chân thành nhất tới thầy Nguyễn Quốc Uy đã quan tâm, giảng dạy và hướng dẫn tận tình, đầy tâm huyết trong suốt quá trình học tập môn Phát triển ứng dụng IoT. Nhờ sự hướng dẫn của thầy, em đã tích lũy được nhiều kiến thức bổ ích và hoàn thành bài tập lớn này. Tuy nhiên, do kiến thức vẫn còn hạn chế, bài tập của em không tránh khỏi những thiếu sót. Em mong thầy xem xét và góp ý để bài báo cáo được hoàn thiện hơn.

Cuối cùng, em xin kính chúc thầy sức khỏe dồi dào, hạnh phúc và thành công trong sự nghiệp trồng người. Chúc thầy luôn truyền lửa đam mê và tiếp tục dìu dắt nhiều thế hệ học trò đến bến bờ tri thức.

Em xin trân trọng cảm ơn!

Sinh viên thực hiện

Phạm Hà Thắng

MỤC LỤC

LỜI CẢM ƠN **1**

MỤC LỤC **2**

DANH MỤC BẢNG BIỂU **5**

DANH MỤC HÌNH ẢNH **6**

CHƯƠNG 1: GIỚI THIỆU CHUNG **8**

1.1. Lý do chọn đề tài **8**

1.2. Mục tiêu của dự án **8**

1.3. Phạm vi và đối tượng nghiên cứu **8**

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT **10**

2.1. Internet of Things (IoT) **10**

2.1.1. Tổng quan về IoT 10

2.1.2. Kiến trúc hệ thống IoT 10

2.2. Giao thức truyền thông MQTT **11**

2.2.1. Tổng quan về MQTT 11

2.2.2. Các thành phần chính của MQTT 11

2.2.3. Ưu điểm của MQTT 12

2.3. Nền tảng phần cứng (Hardware) **12**

2.3.1. Vi điều khiển ESP32 DevKit V1 12

2.3.2. Cảm biến nhiệt độ và độ ẩm DHT11 13

2.3.3. Cảm biến độ ẩm đất (Soil Moisture Sensor) 14

2.3.4. Cảm biến ánh sáng – Quang trở 4 chân (LDR) 15

2.3.5. LED 16

2.4. Công nghệ sử dụng (PERNStack) **17**

2.4.1. Hệ quản trị cơ sở dữ liệu PostgreSQL 17

2.4.2. Node.js và Express.js 17
2.4.3. ReactJS 18
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 20
3.1. Phân tích yêu cầu hệ thống 20
3.2. Kiến trúc hệ thống 20
3.3. Thiết kế phần cứng 21
3.4. Cấu trúc Topic và Message Format 22
3.5. Thiết kế cơ sở dữ liệu 23
3.5.1. Mô tả các bảng 23
3.5.1.1. Bảng Sensor 23
3.5.1.2. Bảng SensorData 23
3.5.1.3. Bảng Device 23
3.5.1.4. Bảng ActivityLog 24
3.5.2. Mối quan hệ giữa các bảng 24
3.5.3. Mô hình ERD 25
3.6. Thiết kế kiến trúc API 25
3.6.1. Quy chuẩn chung 25
3.6.2. Danh sách API chi tiết 26
3.7. Thiết kế sơ đồ tuần tự 26
3.7.1. Luồng đồng bộ dữ liệu thiết bị 26
3.7.2. Luồng thu thập & hiển thị dữ liệu cảm biến và thiết bị 28
3.7.3. Luồng điều khiển thiết bị 29
3.7.4. Luồng xem lịch sử dữ liệu cảm biến 32
3.7.5. Luồng xem lịch sử hành động 34
CHƯƠNG 4: XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG 37

4.1. Triển khai phần cứng 37

4.1.1. Triển khai sơ đồ phần cứng.....37

4.1.2. Lập trình phần cứng: 37

4.2. Giao diện người dùng 42

4.2.1. Giao diện Dashboard.....42

4.2.2. Giao diện Sensor Data 42

4.2.3. Giao diện Action History 43

4.2.4. Giao diện Profile 44

4.3. API Documentation (Postman).....45

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....48

5.1. Kết luận.....48

5.2. Hạn chế 48

5.3. Hướng phát triển 48

DANH MỤC BẢNG BIỂU

Bảng 1. Bảng Sensor 23

Bảng 2. Bảng SensorData 23

Bảng 3. Bảng Device 24

Bảng 4. Bảng ActivityLog 24

Bảng 5. Danh sách API chi tiết 26

DANH MỤC HÌNH ẢNH

Hình 1. ESP32 Dev Kit V1 13
Hình 2. DHT11 14
Hình 3. Cảm biến độ ẩm đất 15
Hình 4. Cảm biến ánh sáng 16
Hình 5. LED 17
Hình 6. Kiến trúc hệ thống 21
Hình 7. Sơ đồ đấu nối phần cứng 22
Hình 8. Mô hình ERD 25
Hình 9. Luồng đồng bộ dữ liệu thiết bị 27
Hình 10. Luồng thu thập & hiển thị dữ liệu cảm biến và thiết bị 29
Hình 11. Luồng điều khiển thiết bị 32
Hình 12. Luồng xem lịch sử dữ liệu cảm biến 34
Hình 13. Luồng xem lịch sử hành động 36
Hình 14. Sơ đồ đấu nối phần cứng thực tế 37
Hình 15. Code cấu hình thư viện và khởi tạo hệ thống 38
Hình 16. Hàm reconnect() để tự động kết nối 39
Hình 17. Hàm callback() xử lý message nhận từ Backend 40
Hình 18. Code đọc, đóng gói và gửi dữ liệu cảm biến cho Backend 41
Hình 19. Giao diện Dashboard 42
Hình 20. Giao diện Sensor Data 43
Hình 21. Giao diện Action History 44
Hình 22. Giao diện Profile 45
Hình 23. API Documentation (1) 45
Hình 24. API Documentation (2) 46

Hình 25. API Documentation (3).....46

Hình 26. API Documentation (4).....46

Hình 27. API Documentation (5).....47

Hình 28. API Documentation (6).....47

CHƯƠNG 1: GIỚI THIỆU CHUNG

1.1. Lý do chọn đề tài

Trong kỷ nguyên Công nghiệp 4.0, Internet of Things (IoT) đang tạo ra những thay đổi mang tính cách mạng trong nhiều lĩnh vực, đặc biệt là nông nghiệp thông minh. Việc áp dụng công nghệ vào quy trình chăm sóc cây trồng không chỉ giúp giảm thiểu công sức lao động thủ công mà còn nâng cao năng suất và chất lượng sản phẩm thông qua việc kiểm soát chính xác các điều kiện môi trường.

Tuy nhiên, các giải pháp nông nghiệp thông minh hiện nay thường có chi phí cao hoặc khó tiếp cận với quy mô hộ gia đình vừa và nhỏ. Xuất phát từ thực tế đó, dự án "Xây dựng hệ thống quản lý thiết bị IOT cho vườn thông minh" được phát triển nhằm cung cấp một giải pháp giám sát và điều khiển tự động với chi phí hợp lý, giao diện trực quan và khả năng phản hồi thời gian thực.

1.2. Mục tiêu của dự án

Dự án tập trung vào việc xây dựng một hệ thống IoT hoàn chỉnh từ phần cứng đến phần mềm, hướng tới các mục tiêu cụ thể sau:

- **Giám sát thời gian thực:** Thu thập và hiển thị liên tục các thông số môi trường quan trọng như nhiệt độ, độ ẩm không khí, độ ẩm đất và cường độ ánh sáng.
- **Điều khiển từ xa:** Cho phép người dùng bật/tắt các thiết bị (máy bơm, đèn, quạt) thông qua giao diện Web ở bất kỳ đâu.
- **Quản lý & Lưu trữ dữ liệu:** Lưu trữ lịch sử hoạt động và dữ liệu cảm biến để phục vụ nhu cầu tra cứu, thống kê và phân tích xu hướng môi trường.
- **Giao diện thân thiện:** Cung cấp trải nghiệm người dùng tối ưu với Dashboard trực quan, phản hồi nhanh và tính tương tác cao.

1.3. Phạm vi và đối tượng nghiên cứu

Đối tượng nghiên cứu: Các giao thức truyền thông IoT (MQTT), lập trình vi điều khiển (ESP32), và phát triển ứng dụng Web.

Phạm vi công nghệ:

- **Phần cứng:** Vi điều khiển ESP32, các cảm biến môi trường (DHT11/DHT22, cảm biến đất, quang trở).
- **Giao thức truyền thông:** Sử dụng MQTT (Message Queuing Telemetry Transport) để đảm bảo tốc độ truyền tin nhanh và ổn định giữa thiết bị và máy chủ.
- **Phần mềm:** Ứng dụng kiến trúc PERN Stack (PostgreSQL, Express.js, ReactJS, Node.js) để xây dựng hệ thống Fullstack.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Internet of Things (IoT)

2.1.1. Tổng quan về IoT

Internet of Things (IoT) hay còn gọi là "Vạn vật kết nối" là một mô hình mạng lưới trong đó các thiết bị vật lý được kết nối với nhau thông qua Internet nhằm thu thập, trao đổi và xử lý dữ liệu một cách tự động. Các thiết bị này có thể là cảm biến, thiết bị gia dụng, phương tiện giao thông hoặc hệ thống công nghiệp, được tích hợp các thành phần như vi điều khiển, phần mềm, cảm biến và các module truyền thông.

IoT cho phép con người giám sát và điều khiển các thiết bị từ xa, đồng thời tối ưu hóa hoạt động thông qua việc phân tích dữ liệu theo thời gian thực. Nhờ đó, IoT đang được ứng dụng rộng rãi trong nhiều lĩnh vực như: nhà thông minh, nông nghiệp thông minh, thành phố thông minh, và y tế thông minh.

Một hệ thống IoT thường có các đặc điểm chính sau:

- **Tính kết nối (Connectivity):** Các thiết bị có khả năng kết nối và trao đổi dữ liệu liên tục.
- **Tính thông minh (Intelligence):** Dữ liệu thu thập được có thể được xử lý để đưa ra quyết định.
- **Tính tự động hóa (Automation):** Hệ thống có thể hoạt động mà không cần con người can thiệp.
- **Khả năng mở rộng (Scalability):** Dễ dàng thêm thiết bị mới vào hệ thống.
- **Tính thời gian thực (Real-time):** Dữ liệu được xử lý gần như ngay lập tức.

2.1.2. Kiến trúc hệ thống IoT

Một hệ thống IoT cơ bản thường được chia làm 3 lớp chính:

- **Lớp nhận thức (Perception Layer):** Đây là lớp thấp nhất trong kiến trúc IoT, có nhiệm vụ thu thập dữ liệu từ môi trường thực. Bao gồm các cảm biến và cơ cấu chấp hành để nhận dạng môi trường xung quanh và thu thập dữ liệu vật lý (nhiệt độ, độ ẩm, ánh sáng,...). Dữ liệu thu thập được

thường là dữ liệu vật lý dạng analog hoặc digital và sẽ được chuyển đổi thành tín hiệu số để xử lý.

- **Lớp mạng (Network Layer):** Đóng vai trò truyền tải và xử lý dữ liệu từ lớp nhận thức thông qua các giao thức mạng như Wi-Fi, Bluetooth, Zigbee hoặc mạng di động (3G/4G/5G). Ngoài ra, lớp này còn đảm nhận việc định tuyến, bảo mật và đảm bảo tính toàn vẹn của dữ liệu trong quá trình truyền.
- **Lớp ứng dụng (Application Layer):** Đây là lớp cao nhất, cung cấp các dịch vụ cụ thể cho người dùng cuối dựa trên dữ liệu thu thập được, ví dụ như nhà thông minh, nông nghiệp thông minh, y tế thông minh.

2.2. Giao thức truyền thông MQTT

2.2.1. Tổng quan về MQTT

MQTT (Message Queuing Telemetry Transport) là một giao thức truyền tải tin nhắn dạng xuất bản/đăng ký (Publish/Subscribe), hoạt động trên nền tảng TCP/IP. Nó được thiết kế để kết nối các thiết bị ở vùng sâu vùng xa với băng thông mạng hạn chế và tiết kiệm năng lượng tối đa cho các vi điều khiển.

Khác với giao thức HTTP hoạt động theo mô hình Request/Response, MQTT tách biệt hoàn toàn người gửi (Publisher) và người nhận (Subscriber) thông qua một máy chủ trung tâm gọi là Broker.

- **Publisher:** Thiết bị gửi dữ liệu (ví dụ: ESP32 gửi dữ liệu cảm biến)
- **Subscriber:** Thiết bị nhận dữ liệu (ví dụ: server hoặc ứng dụng web)
- **Broker:** Trung tâm điều phối dữ liệu

2.2.2. Các thành phần chính của MQTT

MQTT bao gồm các thành phần chính sau:

- **MQTT Broker:** Broker là thành phần quan trọng nhất trong hệ thống MQTT. Nó chịu trách nhiệm nhận mọi tin nhắn, dữ liệu từ Publisher, phân loại dữ liệu theo Topic và gửi tin nhắn, dữ liệu đến tất cả các Subscriber phù hợp. Một số Broker phổ biến như: Mosquitto, HiveMQ, EMQX.
- **MQTT Client:** Client là các thiết bị hoặc ứng dụng kết nối đến Broker. Một client có thể vừa Publish (Gửi) dữ liệu, vừa Subscribe (Nhận) dữ liệu.

- **Topic:** Là một chuỗi định danh được Broker sử dụng để lọc và định tuyến tin nhắn cho mỗi Client kết nối. Topic có cấu trúc phân cấp (ví dụ: garden/sensordata hay garden/device). Topic giúp hệ thống dễ dàng mở rộng và quản lý dữ liệu.

2.2.3. Ưu điểm của MQTT

So với HTTP, MQTT có các ưu điểm sau:

- Header rất nhỏ (chỉ khoảng 2 bytes), giúp giảm tải băng thông mạng.
- Độ trễ thấp → phù hợp thời gian thực
- Tiết kiệm năng lượng → phù hợp thiết bị nhúng
- Hỗ trợ QoS (Quality of Service) với 3 mức độ đảm bảo truyền tin

2.3. Nền tảng phần cứng (Hardware)

2.3.1. Vi điều khiển ESP32 DevKit V1

ESP32 DevKit V1 là một bo mạch phát triển dựa trên chip ESP32 do Espressif sản xuất, được sử dụng rộng rãi trong các ứng dụng IoT nhờ tích hợp sẵn Wi-Fi và Bluetooth.

Đặc điểm kỹ thuật:

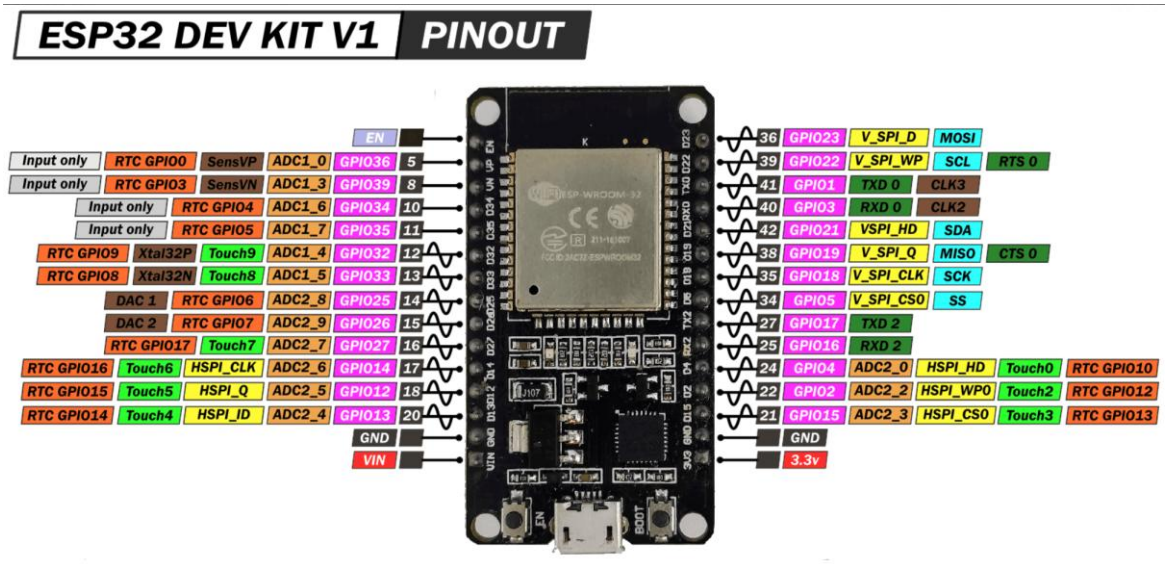
- Vi xử lý: Dual-core Tensilica Xtensa LX6
- Tần số xung nhịp: lên đến 240 MHz
- Bộ nhớ SRAM: ~520 KB
- Flash: thường từ 4MB trở lên
- Tích hợp Wi-Fi 802.11 b/g/n và Bluetooth 4.2
- Số chân GPIO: khoảng 30+ chân
- Hỗ trợ ADC (Analog to Digital Converter), DAC, PWM

Chức năng trong hệ thống:

- Đọc dữ liệu từ các cảm biến (DHT11, độ ẩm đất, ánh sáng)
- Xử lý dữ liệu sơ bộ
- Gửi dữ liệu lên server thông qua giao thức MQTT
- Nhận lệnh điều khiển từ server (ví dụ bật/tắt máy bơm)

Ưu điểm:

- Tích hợp nhiều chức năng trong một chip
- Giá thành thấp
- Tiêu thụ điện năng thấp
- Cộng đồng hỗ trợ lớn



Hình 1. ESP32 Dev Kit V1

2.3.2. Cảm biến nhiệt độ và độ ẩm DHT11

DHT11 là cảm biến kỹ thuật số dùng để đo nhiệt độ và độ ẩm không khí, được sử dụng phổ biến trong các dự án IoT cơ bản.

Thông số kỹ thuật:

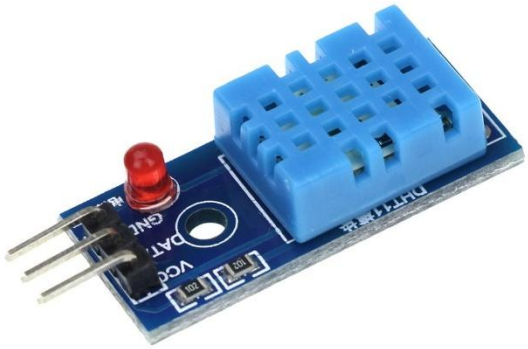
- Điện áp hoạt động: 3.3V – 5V
- Phạm vi đo nhiệt độ: 0°C đến 50°C (sai số ±2°C)
- Phạm vi đo độ ẩm: 20% – 90% RH (sai số ±5%)
- Tần số lấy mẫu: 1Hz (1 lần/giây)

Nguyên lý hoạt động:

- DHT11 sử dụng:
  - Một cảm biến độ ẩm điện trở
  - Một cảm biến nhiệt điện trở
- Dữ liệu được chuyển đổi sang dạng tín hiệu số và truyền về ESP32 thông qua giao tiếp 1 dây.

Vai trò trong hệ thống:

- Giám sát điều kiện môi trường không khí
- Cung cấp dữ liệu để hiển thị và phân tích



Hình 2. DHT11

2.3.3. Cảm biến độ ẩm đất (Soil Moisture Sensor)

Cảm biến độ ẩm đất được sử dụng để đo mức độ ẩm của đất, rất quan trọng trong các hệ thống nông nghiệp thông minh.

Cấu tạo:

- Hai đầu dò kim loại cắm vào đất
- Module xử lý tín hiệu (có thể xuất analog hoặc digital)

Nguyên lý hoạt động:

- Độ ẩm đất ảnh hưởng đến độ dẫn điện
- Đất càng ẩm → dẫn điện càng tốt → giá trị điện áp thay đổi
- ESP32 đọc giá trị analog từ cảm biến

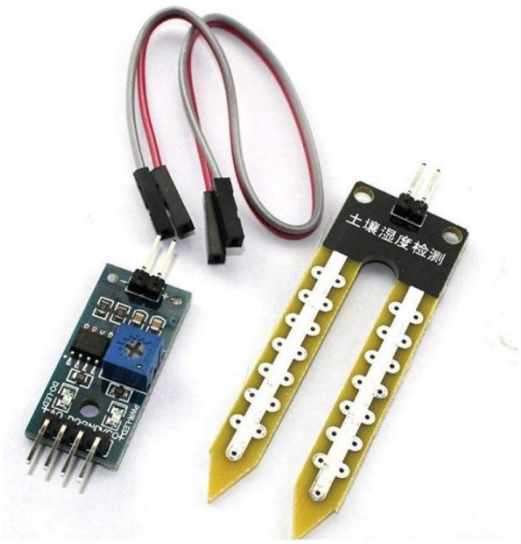
Dạng tín hiệu:

- Analog (giá trị liên tục)
- Digital (ngưỡng có/không)

Vai trò trong hệ thống:

- Giám sát điều kiện đất

- Cung cấp dữ liệu để hiển thị và phân tích



Hình 3. Cảm biến độ ẩm đất

2.3.4. Cảm biến ánh sáng – Quang trở 4 chân (LDR)

Quang trở (Light Dependent Resistor – LDR) là một loại điện trở có giá trị thay đổi theo cường độ ánh sáng.

Đặc điểm:

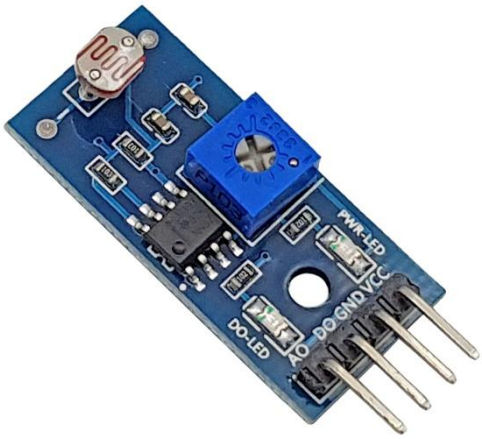
- Điện trở giảm khi ánh sáng tăng
- Điện trở tăng khi ánh sáng giảm

Nguyên lý hoạt động:

- Khi có ánh sáng chiếu vào, các electron được giải phóng → tăng độ dẫn điện
- ESP32 đọc giá trị điện áp thông qua mạch chia điện áp

Ứng dụng trong hệ thống:

- Xác định ngày/đêm
- Điều khiển đèn tự động
- Theo dõi cường độ ánh sáng môi trường



Hình 4. Cảm biến ánh sáng

2.3.5. LED

LED (Light Emitting Diode) là một linh kiện bán dẫn hai chân, được sử dụng phổ biến trong các mạch điện tử và hệ thống IoT nhờ khả năng phát sáng hiệu quả và tiêu thụ điện năng thấp. LED gồm hai chân: Anode (chân dương) và Cathode (chân âm), được bọc trong lớp vỏ nhựa epoxy giúp bảo vệ và khuếch tán ánh sáng.

Khi có dòng điện chạy theo chiều thuận từ Anode đến Cathode, LED phát sáng do quá trình tái hợp electron trong vật liệu bán dẫn. Điện áp hoạt động của LED thường nằm trong khoảng từ 1.8V đến 3.3V, với dòng điện định mức khoảng 20mA. Để đảm bảo an toàn và tránh hư hỏng, LED luôn cần được mắc nối tiếp với một điện trở hạn dòng phù hợp.

LED có thể phát ra nhiều màu sắc khác nhau như đỏ, xanh lá, xanh dương hoặc trắng, tùy thuộc vào vật liệu cấu tạo. Nhờ ưu điểm như tuổi thọ cao, kích thước nhỏ gọn và phản hồi nhanh, LED thường được sử dụng trong các hệ thống để hiển thị trạng thái hoạt động.

Trong các dự án IoT, LED thường được điều khiển thông qua chân GPIO của vi điều khiển, sử dụng mức logic HIGH/LOW để bật hoặc tắt, hoặc điều chế PWM để thay đổi độ sáng. Khi thiết kế mạch, cần chú ý đến cực tính và giới hạn dòng điện để đảm bảo LED hoạt động ổn định và bền bỉ.

Hình 5. LED

2.4. Công nghệ sử dụng (PERNStack)

2.4.1. Hệ quản trị cơ sở dữ liệu PostgreSQL

PostgreSQL là một hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, được đánh giá cao về tính ổn định, khả năng mở rộng và tuân thủ chuẩn SQL. Trong hệ thống IoT, dữ liệu thu thập từ các cảm biến thường có tính liên tục theo thời gian và cần được lưu trữ một cách có cấu trúc để phục vụ cho việc truy vấn, phân tích và hiển thị.

Việc sử dụng PostgreSQL cho phép hệ thống lưu trữ dữ liệu cảm biến như nhiệt độ, độ ẩm, ánh sáng theo dạng bảng với các trường thời gian (timestamp), từ đó hỗ trợ việc truy vấn dữ liệu theo khoảng thời gian hoặc theo từng thiết bị cụ thể. Ngoài ra, PostgreSQL còn hỗ trợ mạnh mẽ các truy vấn phức tạp, giúp thực hiện các phép thống kê như trung bình, giá trị lớn nhất/nhỏ nhất hoặc phân tích xu hướng môi trường theo thời gian.

Một ưu điểm nổi bật khác là khả năng đảm bảo tính toàn vẹn dữ liệu thông qua các cơ chế như ràng buộc (constraints), khóa chính (primary key) và khóa ngoại (foreign key). Điều này giúp hệ thống tránh được các lỗi dữ liệu không nhất quán, đặc biệt quan trọng khi số lượng thiết bị và dữ liệu ngày càng tăng.

2.4.2. Node.js và Express.js

Phía backend của hệ thống được xây dựng dựa trên Node.js kết hợp với framework Express.js. Node.js là môi trường thực thi JavaScript phía máy chủ, cho phép xử lý các tác vụ bất đồng bộ (asynchronous) một cách hiệu quả. Điều này đặc biệt phù

hợp với hệ thống IoT, nơi dữ liệu từ các thiết bị được gửi liên tục và cần được xử lý gần như theo thời gian thực.

Nhờ mô hình non-blocking I/O, Node.js có thể xử lý đồng thời nhiều kết nối mà không gây tắc nghẽn, giúp hệ thống duy trì hiệu năng ổn định ngay cả khi có nhiều thiết bị ESP32 gửi dữ liệu cùng lúc. Đây là một lợi thế lớn so với các mô hình xử lý đồng bộ truyền thống.

Express.js đóng vai trò là framework hỗ trợ xây dựng các API một cách nhanh chóng và có tổ chức. Thông qua Express, hệ thống backend có thể cung cấp các endpoint RESTful để:

- Nhận dữ liệu từ MQTT hoặc từ thiết bị
- Lưu dữ liệu vào PostgreSQL
- Cung cấp dữ liệu cho frontend
- Xử lý các yêu cầu điều khiển thiết bị

Ngoài ra, backend còn đóng vai trò trung gian giữa MQTT Broker và ứng dụng web, giúp đồng bộ dữ liệu giữa thiết bị IoT và giao diện người dùng. Điều này đảm bảo rằng mọi thay đổi từ cảm biến hoặc từ người dùng đều được cập nhật kịp thời trong toàn hệ thống.

2.4.3. ReactJS

Phía frontend của hệ thống được phát triển bằng ReactJS, một thư viện JavaScript mạnh mẽ do Meta phát triển, chuyên dùng để xây dựng giao diện người dùng tương tác cao. ReactJS cho phép xây dựng giao diện theo hướng component, giúp chia nhỏ hệ thống thành các phần độc lập, dễ quản lý và tái sử dụng.

Trong bối cảnh hệ thống IoT, ReactJS đóng vai trò hiển thị dữ liệu cảm biến theo thời gian thực, đồng thời cung cấp các chức năng điều khiển thiết bị từ xa cho người dùng. Ví dụ, người dùng có thể theo dõi nhiệt độ, độ ẩm hoặc trạng thái thiết bị thông qua dashboard trực quan, và thực hiện các thao tác như bật/tắt máy bơm hoặc đèn chỉ với một thao tác trên giao diện web.

Một trong những ưu điểm nổi bật của ReactJS là khả năng cập nhật giao diện nhanh chóng mà không cần tải lại toàn bộ trang (Single Page Application – SPA). Điều

này giúp cải thiện trải nghiệm người dùng, đặc biệt trong các hệ thống yêu cầu cập nhật dữ liệu liên tục như IoT.

Ngoài ra, ReactJS có thể dễ dàng tích hợp với các công nghệ realtime như WebSocket hoặc MQTT thông qua backend, giúp hiển thị dữ liệu gần như ngay lập tức khi có sự thay đổi từ thiết bị.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu hệ thống

Để xây dựng một hệ thống IoT hoàn chỉnh và có khả năng ứng dụng thực tế, hệ thống cần đáp ứng các yêu cầu khắt khe về mặt chức năng và phi chức năng:

Yêu cầu chức năng:

- Hệ thống phần cứng phải đọc được dữ liệu từ các cảm biến với chu kỳ 2 giây/lần và gửi lên máy chủ.
- Máy chủ phải lưu trữ toàn bộ lịch sử dữ liệu này vào cơ sở dữ liệu để phục vụ tra cứu.
- Giao diện Web phải hiển thị dữ liệu thời gian thực trên các thẻ thông số và biểu đồ.
- Người dùng có thể điều khiển Bật/Tắt 4 thiết bị mô phỏng trực tiếp trên giao diện.
- Có chức năng tra cứu "Dữ liệu cảm biến" và "Lịch sử hoạt động" với khả năng phân trang, lọc, sắp xếp và tìm kiếm thông minh.

Yêu cầu phi chức năng:

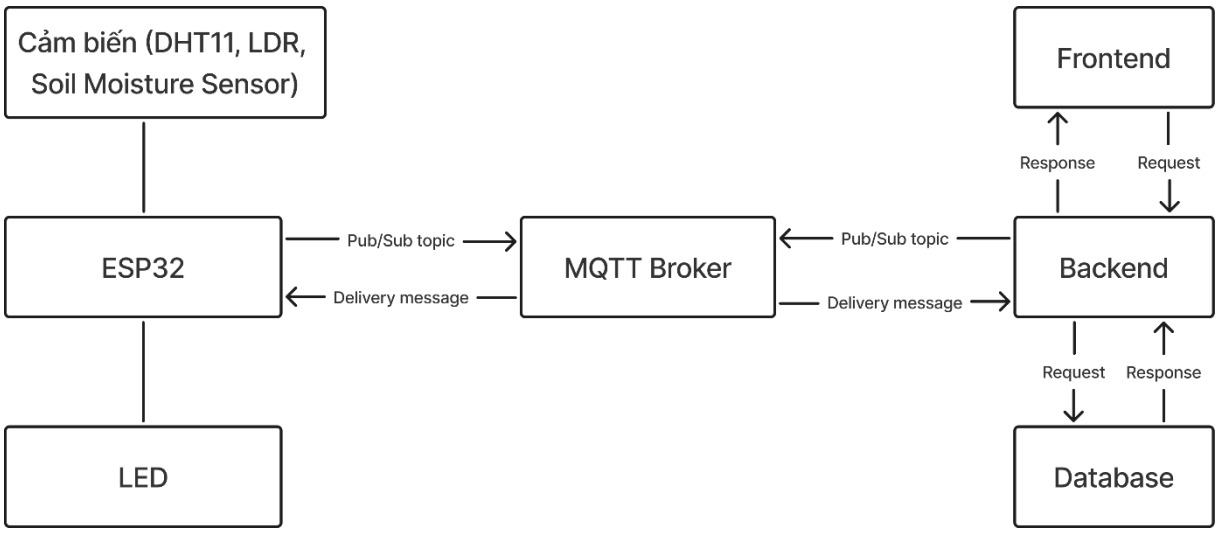
- **Độ trễ thấp:** Thao tác điều khiển thiết bị phải được thực thi ngay lập tức.
- **Độ tin cậy:** Phải có cơ chế xử lý lỗi nếu phần cứng mất kết nối và cơ chế đồng bộ lại trạng thái khi có điện trở lại.
- **Hiệu năng:** Truy vấn dữ liệu không được làm giật lag máy chủ khi dữ liệu phình to lên hàng triệu bản ghi.

3.2. Kiến trúc hệ thống

Kiến trúc hệ thống được thiết kế theo mô hình phân lớp, đảm bảo tính module hóa cao, dễ dàng mở rộng và bảo trì. Hệ thống gồm 3 lớp chính:

- **Lớp Thiết bị:** Sử dụng vi điều khiển ESP32 làm bộ xử lý trung tâm. Thiết bị này trực tiếp giao tiếp với các cảm biến (để đo nhiệt độ, độ ẩm, ánh sáng) và điều khiển các cơ cấu chấp hành (bơm nước, quạt, đèn - được mô phỏng bằng LED).

- **Lớp truyền thông:** Sử dụng giao thức MQTT đóng vai trò là cầu nối thời gian thực giữa phần cứng và Backend.
- **Lớp Nghiệp vụ:** Backend được xây dựng bằng Node.js/Express với kiến trúc RESTful API, tích hợp cơ sở dữ liệu PostgreSQL để lưu trữ dữ liệu và cung cấp các dịch vụ xử lý dữ liệu phức tạp. Nó vừa đóng vai trò là MQTT Client để liên tục lắng nghe/gửi bản tin từ MQTT Broker, vừa cung cấp các RESTful API cho ứng dụng Web. Lớp này thực hiện các logic nghiệp vụ quan trọng.
- **Lớp Ứng dụng:** Ứng dụng Web Single Page Application (SPA) xây dựng bằng ReactJS, trực quan hóa dữ liệu thông qua các Dashboard, Data Table và cung cấp bảng điều khiển cho người dùng cuối.

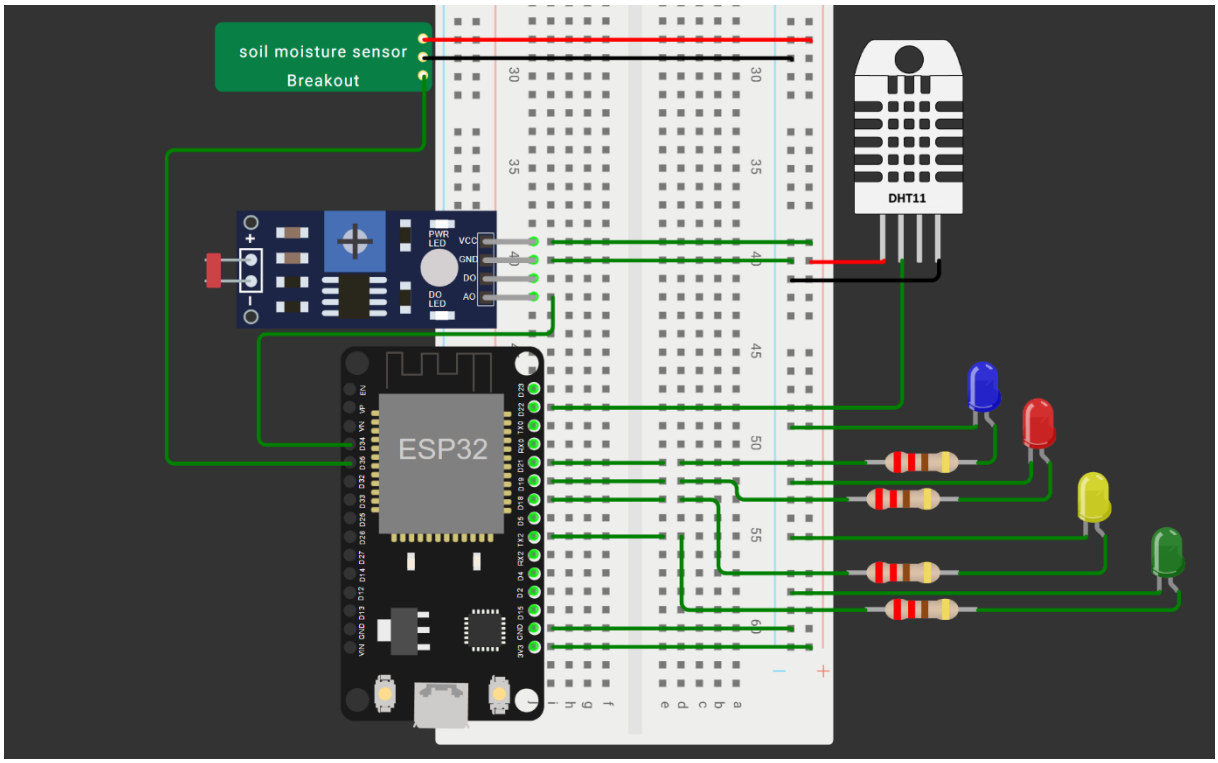


Hình 6. Kiến trúc hệ thống

3.3. Thiết kế phần cứng

➢ **Sơ đồ đấu nối:**

- Cảm biến nhiệt độ, độ ẩm DHT11 được nối với chân Digital (GPIO 22).
- Cảm biến độ ẩm đất và quang trở (LDR) trả về tín hiệu liên tục, được nối vào các chân ADC tương ứng là GPIO 35 và GPIO 34.
- 4 đèn LED (Xanh lá, Vàng, Đỏ, Xanh dương) nối với các chân GPIO 17, 18, 19, 21 thông qua điện trở hạn dòng 220Ω để bảo vệ vi điều khiển.



Hình 7. Sơ đồ đấu nối phần cứng

3.4. Cấu trúc Topic và Message Format

Hệ thống sử dụng cấu trúc topic phân cấp như sau:

- garden/sensordata: Chứa dữ liệu cảm biến được gửi từ ESP32
- garden/devicecontrol: Chứa lệnh điều khiển bật tắt thiết bị được gửi từ Backend
- garden/status: Chứa phản hồi sau khi bật tắt thiết bị
- garden/sync: Chứa lệnh yêu cầu đồng bộ thiết bị

Các Message được gửi dưới dạng như sau:

- Dữ liệu cảm biến được gửi dưới dạng JSON với các trường: temp (Nhiệt độ), humid (Độ ẩm không khí), light (Ánh sáng), soil (Độ ẩm đất). VD: { "temp": 28, "humid": 65, "light": 540, "soil": 27.5 }
- Lệnh điều khiển LED được gửi dưới dạng JSON với các trường: device_type (Loại thiết bị), action (Hành động). VD: { "device_type": "fan", "action": "ON"}
- Phản hồi lệnh điều khiển được gửi dưới dạng JSON với các trường: device_type (Loại thiết bị), action (Hành động), status (Trạng thái). VD: { "device_type": "fan", "action": "ON", "status": "SUCCESS" }.

3.5. Thiết kế cơ sở dữ liệu

3.5.1. Mô tả các bảng

3.5.1.1. Bảng Sensor

Lưu thông tin của cảm biến.

Tên trường	Kiểu dữ liệu	Bắt buộc	Mô tả & Ví dụ
id	varchar (255)	Có	Khóa chính (Primary Key).
name	varchar (255)	Có	Tên của cảm biến được hiển thị trên hệ thống.VD: "Cảm biến Nhiệt độ", "Độ ẩm đất".
unit	varchar (255)	Có	Đơn vị đo. VD: “%”, “°C”
type	varchar (255)	Có	Dùng để định danh trong Topic MQTT. VD: “humid”, “temp”
slug	varchar(255)	Có	Đường dẫn thân thiện
created_at	date	Có	Thời gian tạo

Bảng 1. Bảng Sensor

3.5.1.2. Bảng SensorData

Lưu trữ mọi dữ liệu cảm biến.

Tên trường	Kiểu dữ liệu	Bắt buộc	Mô tả & Ví dụ
id	varchar (255)	Có	Khóa chính (Primary Key).
sensor_id	varchar (255)	Có	Khóa ngoại (FK): Liên kết với bảng Sensors.
value	float	Có	Giá trị đo được của cảm biến
created_at	date	Có	Thời điểm ghi nhận giá trị.

Bảng 2. Bảng SensorData

3.5.1.3. Bảng Device

Lưu thông tin và trạng thái hiện tại của thiết bị

Tên trường	Kiểu dữ liệu	Bắt buộc	Mô tả & Ví dụ
id	varchar (255)	Có	Khóa chính (Primary Key).
name	varchar (255)	Có	Tên thiết bị.
type	varchar (255)	Có	Dùng để định danh trong Topic MQTT.
status	varchar (255)	Có	Trạng thái của thiết bị

slug	varchar(255)	Có	Đường dẫn thân thiện
created_at	date	Có	Thời gian tạo

Bảng 3. Bảng Device

3.5.1.4. Bảng ActivityLog

Lưu vết lịch sử chuyển trạng thái của thiết bị.

Tên trường	Kiểu dữ liệu	Bắt buộc	Mô tả & Ví dụ
id	varchar (255)	Có	Khóa chính (Primary Key).
device_id	varchar (255)	Có	Khóa ngoại (FK): Liên kết với bảng Devices.
action	varchar (255)	Có	Hành động thực hiện
status	varchar (255)	Có	Trạng thái của hành động
created_at	date	Có	Thời điểm thực hiện hành động

Bảng 4. Bảng ActivityLog

3.5.2. Mối quan hệ giữa các bảng

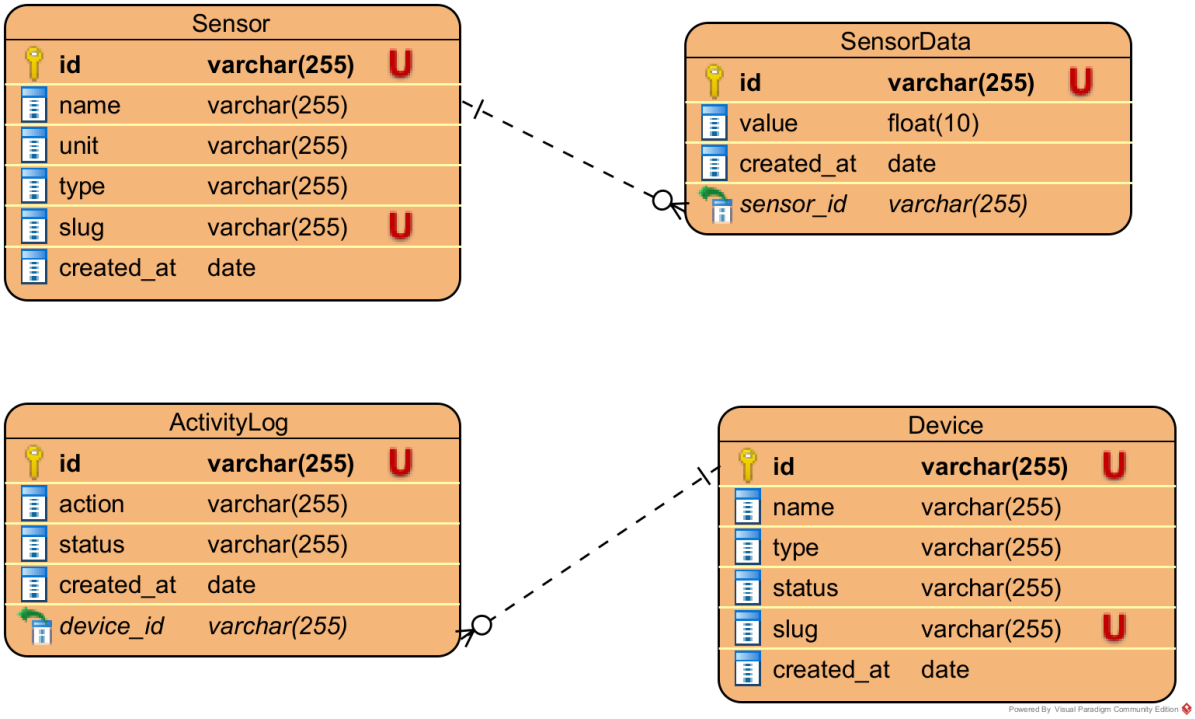
Một Sensor (Cảm biến) có nhiều SensorData (Dữ liệu đo), một SensorData (Dữ liệu đo) phải thuộc về một Sensor.

→ Quan hệ giữa Sensor và SensorData là 1-N.

Một Device (Thiết bị) có nhiều ActivityLog (Lịch sử hành động), một ActivityLog (Lịch sử hành động) phải thuộc về một Device (Thiết bị).

→ Quan hệ giữa Device và ActivityLog là 1-N.

3.5.3. Mô hình ERD



Hình 8. Mô hình ERD

3.6. Thiết kế kiến trúc API

Hệ thống giao tiếp giữa Client và Server thông qua chuẩn RESTful API. Dữ liệu được trao đổi dưới định dạng JSON.

3.6.1. Quy chuẩn chung

Base URL: http://<server-ip>:5000/api

HTTP Methods:

- GET: Truy xuất dữ liệu.
- POST: Tạo mới dữ liệu.
- PUT: Cập nhật dữ liệu.
- DELETE: Xóa dữ liệu.

HTTP Status Codes:

- **200 OK:** Thành công.
- **201 Created:** Tạo mới thành công.
- **400 Bad Request:** Lỗi dữ liệu đầu vào.
- **401 Unauthorized:** Chưa đăng nhập hoặc Token hết hạn.

- **403 Forbidden:** Không có quyền truy cập
- **404 Not Found:** Không tìm thấy tài nguyên.
- **500 Internal Server Error:** Lỗi hệ thống.

3.6.2. Danh sách API chi tiết

Phương thức	Endpoint	Mô tả
GET	/api/dashboard/data	Lấy toàn bộ toàn dữ liệu cần thiết để vẽ màn hình Dashboard (4 thẻ thông số, dữ liệu biểu đồ, và trạng thái 4 công tắc).
GET	/api/devices	Lấy danh sách thiết bị động, phục vụ cho việc render bộ lọc (Filter) bên trang Lịch sử hoạt động.
POST	/api/devices/control	Gửi yêu cầu bật/tắt thiết bị
GET	/api/activity-logs	Lấy danh sách lịch sử bật/tắt thiết bị có hỗ trợ phân trang, tìm kiếm, lọc và sắp xếp.
GET	/api/sensor-datas	Lấy danh sách dữ liệu cảm biến có hỗ trợ phân trang, tìm kiếm, lọc và sắp xếp.
GET	/sensor-datas	Lấy danh sách cảm biến động, phục vụ cho việc render bộ lọc (Filter) bên trang Sensor Data.

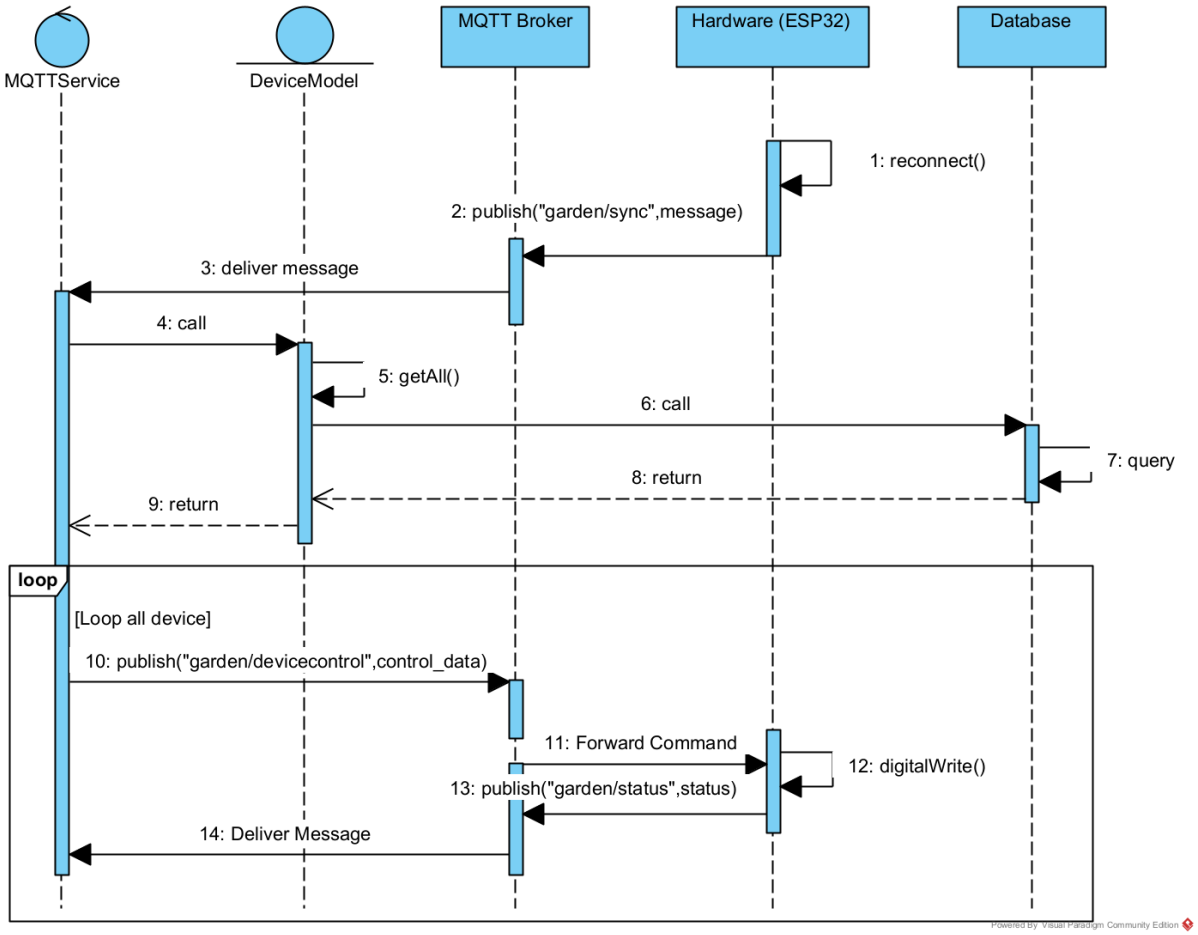
Bảng 5. Danh sách API chi tiết

3.7. Thiết kế sơ đồ tuần tự

3.7.1. Luồng đồng bộ dữ liệu thiết bị

1. Khi ESP32 được cấp nguồn hoặc khôi phục mạng thành công nó sẽ gọi đến hàm reconnect() để thực hiện kết nối MQTT Broker
2. Khi kết nối thành công, ESP32 sẽ gửi (publish) lên topic garden/sync qua MQTT Broker để yêu cầu Backend đồng bộ dữ liệu.
3. MQTT Broker chuyển tiếp dữ liệu đến MQTTService của Backend (Service này đã subscribe topic garden/sync từ trước).
4. MQTTService nhận tin, parse JSON và gọi đến hàm getAll() DeviceModel để lấy trạng thái của thiết bị.
5. Hàm getAll() của DeviceModel được kích hoạt

6. Hàm getAll() của DeviceModel thực hiện kết nối và yêu cầu Database lấy dữ liệu
7. Database thực hiện truy vấn dữ liệu
8. Database trả dữ liệu cho DeviceModel
9. DeviceModel trả dữ liệu cho MQTTService
10. MQTTService thực hiện lặp qua danh sách các thiết bị được trả về từ và gửi lệnh (publish) tới MQTT Broker qua topic "garden/devicecontrol".
11. MQTT Broker chuyển lệnh xuống Hardware (ESP32).
12. Hardware nhận lệnh và xử lý bật/tắt thiết bị (Led).
13. Hardware gửi phản hồi qua MQTT với topic "garden/status".
14. MQTT Broker chuyển tiếp gói tin đến MQTTService của Backend (Service này đã subscribe topic từ khi khởi động).

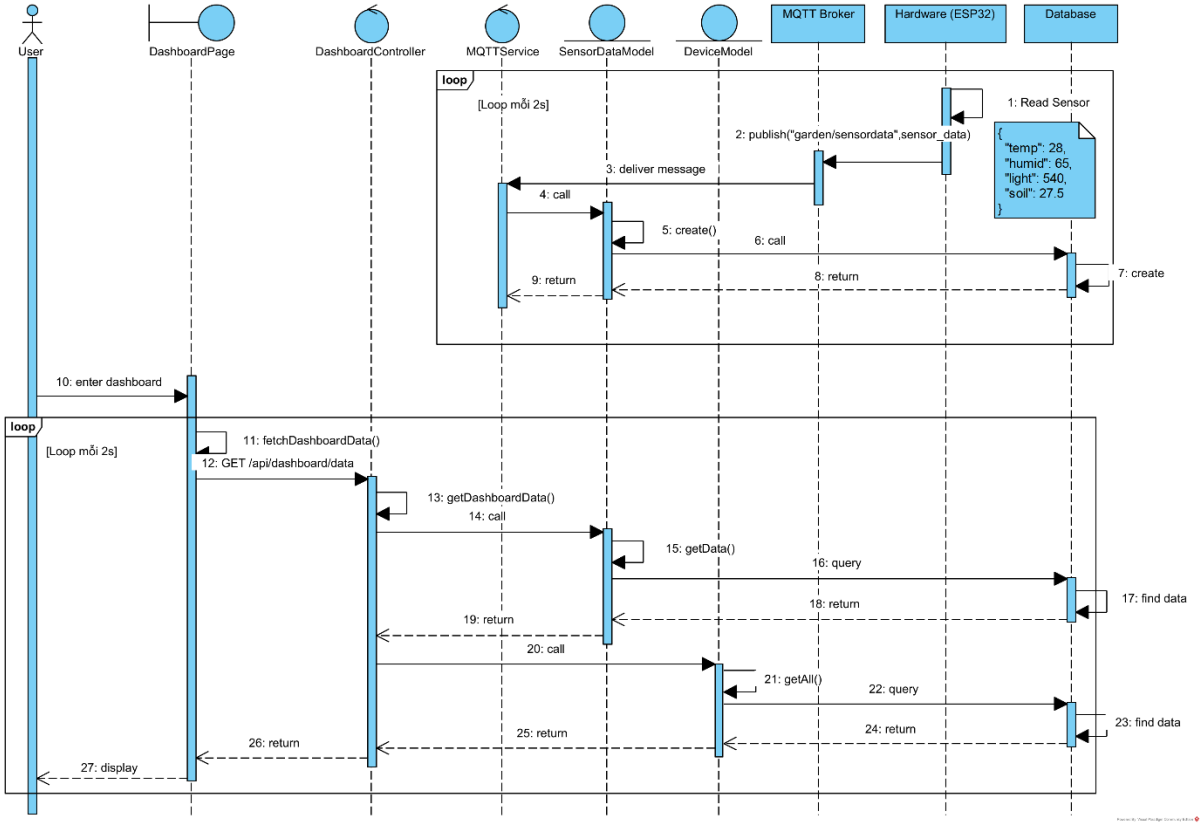


Hình 9. Luồng đồng bộ dữ liệu thiết bị

3.7.2. Luồng thu thập & hiển thị dữ liệu cảm biến và thiết bị

1. Hardware (ESP32) kích hoạt hàm đọc giá trị từ các cảm biến (DHT11, Cảm biến đất, Quang trở) định kỳ 2s/lần.
2. Hardware (ESP32) đóng gói dữ liệu thành JSON và gửi (publish) lên topic garden/sensordata qua MQTT Broker. VD: {"temp": 28, "humid": 65, "light": 540, "soil": 27.5}
3. MQTT Broker chuyển tiếp nguyên vẹn gói tin JSON trên đến MQTTService của Backend (Service này đã subscribe topic garden/sensordata từ trước).
4. MQTTService nhận tin, parse JSON và gọi đến hàm create() của SensorDataModel.
5. Hàm create() của SensorDataModel được kích hoạt.
6. Hàm create() của SensorDataModel thực hiện kết nối và yêu cầu Database lưu dữ liệu.
7. Database lưu các dữ liệu mới.
8. Database trả dữ liệu về cho SensorDataModel.
9. SensorDataModel trả về kết quả xác nhận (Success) cho MQTTService
10. User truy cập vào trang Dashboard
11. DashboardPage khởi chạy vòng lặp cập nhật (Polling) mỗi 2 giây và gọi hàm fetchDashboardData().
12. Hàm fetchDashboardData() được kích hoạt và gửi request tới API GET api/dashboard/data.
13. DashboardController nhận request và kích hoạt hàm getDashboardData().
14. Hàm getDashboardData() của DashboardController gọi đến hàm getData() của SensorDataModel để yêu cầu truy xuất dữ liệu.
15. Hàm getData() của SensorDataModel kích hoạt.
16. Hàm getData() của SensorDataModel thực hiện kết nối và yêu cầu Database lấy dữ liệu
17. Database thực hiện truy vấn dữ liệu.
18. Database trả dữ liệu về cho SensorDataModel.
19. SensorDataModel trả dữ liệu về cho DashboardController
20. Hàm getDashboardData() của DashboardController gọi đến hàm getAll() của DeviceModel để yêu cầu truy xuất dữ liệu.

21. Hàm getAll() của DeviceModel được kích hoạt
22. Hàm getAll() của DeviceModel thực hiện kết nối và yêu cầu Database lấy dữ liệu
23. Database thực hiện truy vấn dữ liệu.
24. Database trả dữ liệu về cho DeviceModel.
25. Deivce trả dữ liệu về cho DashboardController
26. DashboardController xử lý dữ liệu và trả phản hồi về cho DashboardPage.
27. DashboardPage cập nhật giao diện và hiển thị cho User.



Hình 10. Luồng thu thập & hiển thị dữ liệu cảm biến và thiết bị

3.7.3. Luồng điều khiển thiết bị

1. User nhấn nút "Bật/Tắt" thiết bị trên giao diện Dashboard.
2. Dashboard chuyển trạng thái nút bấm sang Loading và gọi hàm toggleDevice().
3. Hàm toggleDevice() gửi request API POST /api/devices/control tới Backend.
4. DeviceController tiếp nhận request và kích hoạt hàm controlDevice().
5. Hàm controlDevice() của DeviceController kích hoạt và gọi hàm create() của ActivityLogModel

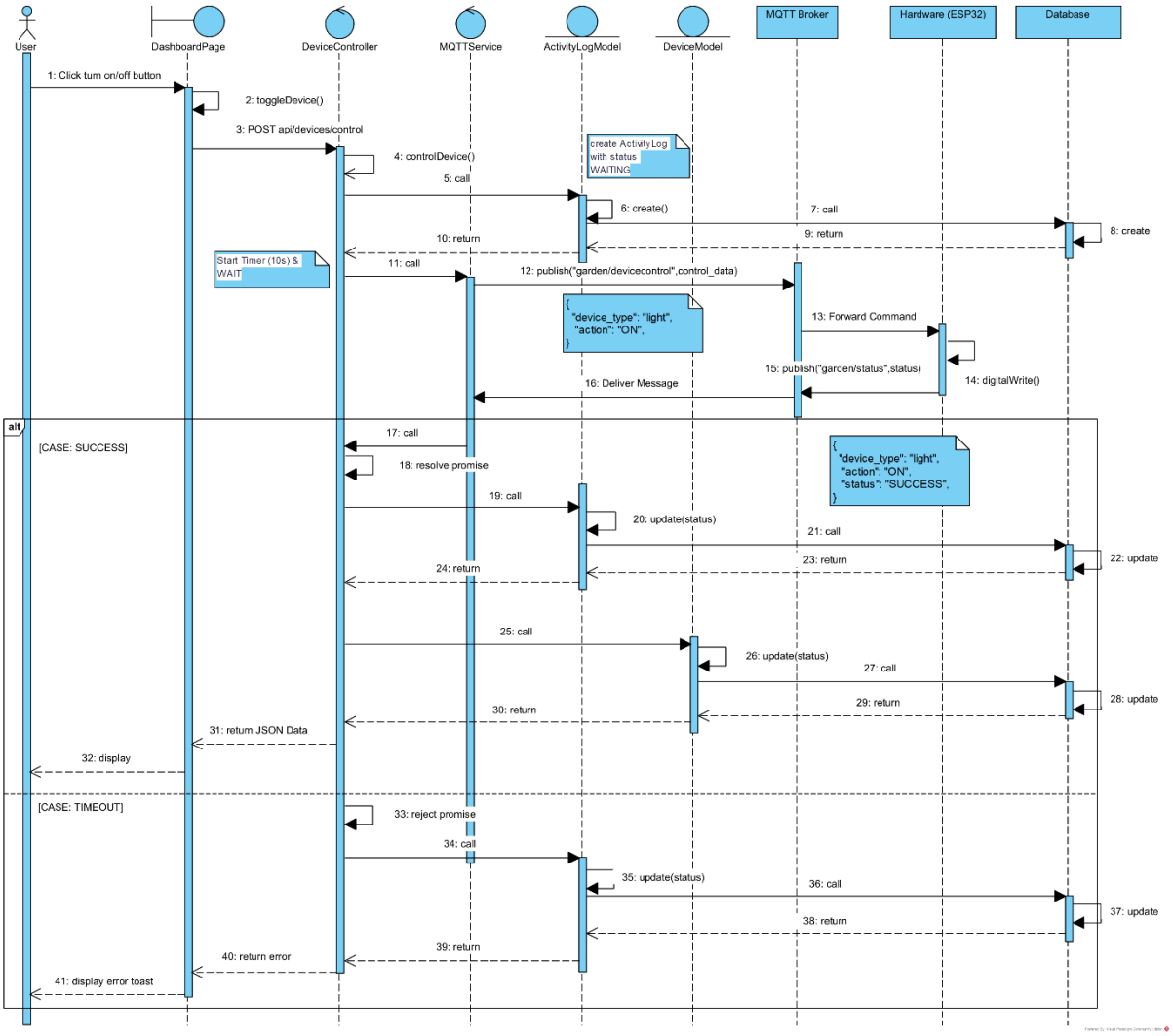
6. Hàm create() của ActivityLogModel kích hoạt
7. Hàm create() của ActivityLogModel thực hiện kết nối với Database và tạo dữ liệu mới (ActivityLog mới có action là hành động của user với status = 'WAITING' để ghi nhận hành động đã bắt đầu).
8. Database thực hiện tạo dữ liệu mới.
9. Database trả dữ liệu về cho ActivityLogModel.
10. ActivityLogModel trả về kết quả cho DeviceController.
11. DeviceController gọi MQTTService, khởi tạo một Promise để lắng nghe sự kiện phản hồi và kích hoạt bộ đếm Timeout 10s.
12. MQTTService gửi lệnh (publish) tới MQTT Broker qua topic "garden/devicecontrol".
13. MQTT Broker chuyển lệnh xuống Hardware (ESP32).
14. Hardware nhận lệnh và xử lý bật/tắt thiết bị (Led).
15. Hardware gửi phản hồi qua MQTT với topic "garden/status".
16. MQTT Broker chuyển tiếp gói tin đến MQTTService của Backend (Service này đã subscribe topic từ khi khởi động).

    [Trường hợp Thành công]
17. MQTTService nhận tin và phát ra tín hiệu cho DeviceController.
18. DeviceController giải quyết Promise mà nó đang chờ.
19. DeviceController gọi đến hàm update() của ActivityLogModel để cập nhật trạng thái.
20. Hàm update() của ActivityLogModel được kích hoạt
21. Hàm update() của ActivityLogModel thực hiện kết nối và yêu cầu Database cập nhật dữ liệu.
22. Database thực hiện cập nhật dữ liệu
23. Database trả dữ liệu về cho ActivityLogModel
24. ActivityLogModel trả về dữ liệu cho DeviceController.
25. DeviceController gọi đến hàm update() của DeviceModel để cập nhật trạng thái.
26. Hàm update() của DeviceModel được kích hoạt
27. Hàm update() của DeviceModel thực hiện kết nối và yêu cầu Database cập nhật dữ liệu.

28. Database thực hiện cập nhật dữ liệu
29. Database trả dữ liệu về cho DeviceModel
30. DeviceModel trả về kết quả xác nhận cho DeviceController.
31. DeviceController xử lý dữ liệu và trả phản hồi về cho DashboardPage.
32. DashboardPage cập nhật giao diện và hiển thị cho User.

[Trường hợp thất bại]

33. (Nếu sau 10s không nhận được tin từ MQTTService như bước 17) DeviceController bắt lỗi Timeout, Reject Promise mà nó đang chờ.
34. DeviceController gọi đến hàm update của ActivityLogModel để cập nhật trạng thái.
35. Hàm update() của ActivityLogModel được kích hoạt
36. Hàm update() của ActivityLogModel thực hiện kết nối và yêu cầu Database cập nhật dữ liệu.
37. Database thực hiện cập nhật dữ liệu
38. Database trả dữ liệu về cho ActivityLogModel
39. ActivityLogModel trả về kết quả xác nhận cho DeviceController.
40. DeviceController trả về lỗi cho DashboardPage.
41. DashboardPage tắt Loading và hiện thông báo lỗi cho User.

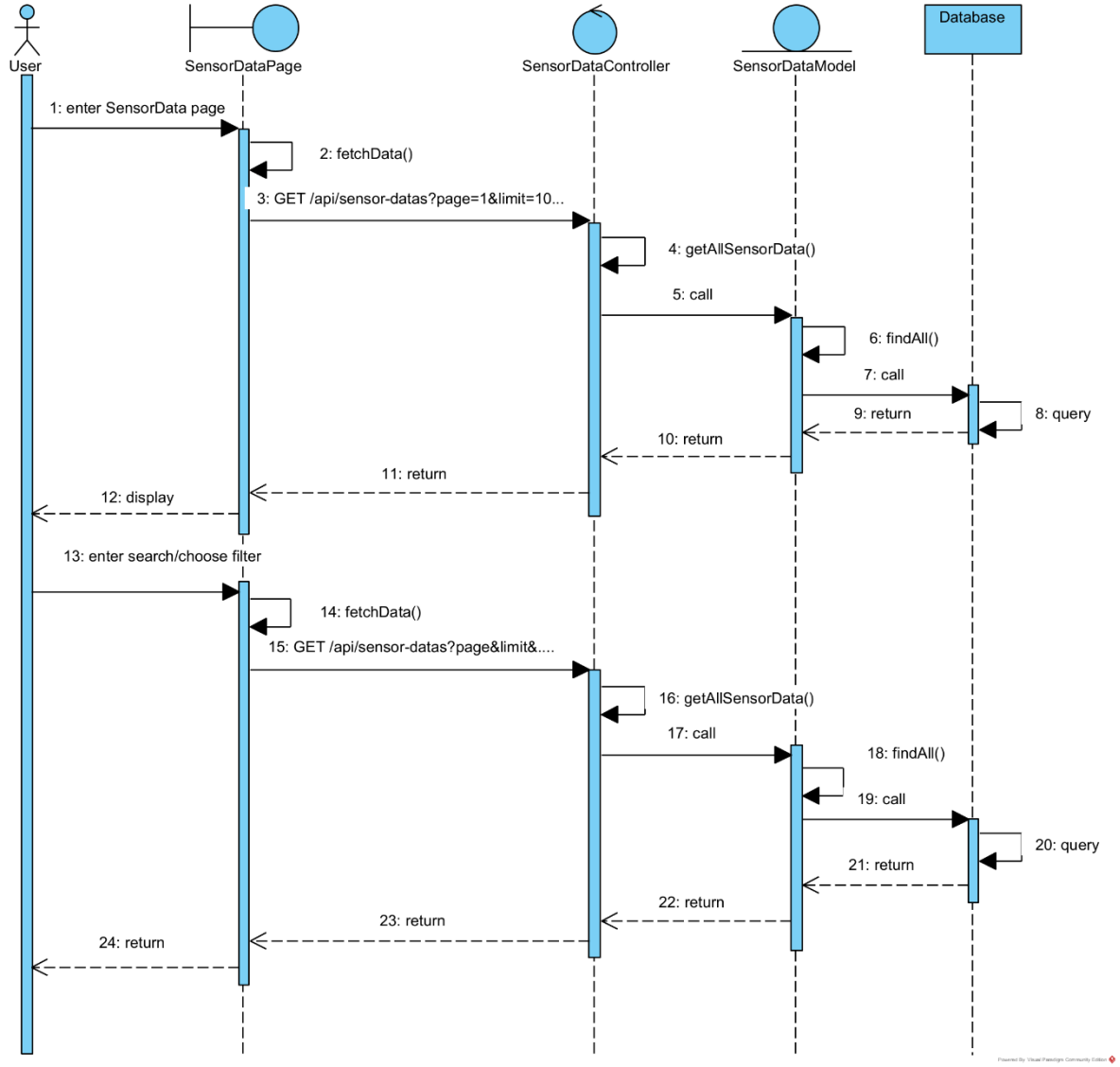


Hình 11. Luồng điều khiển thiết bị

3.7.4. Luồng xem lịch sử dữ liệu cảm biến

1. User truy cập vào trang SensorData.
2. SensorDataPage khởi tạo giao diện và gọi hàm fetchData().
3. Hàm fetchData() kích hoạt và gửi request API GET /api/sensor-datas?page=1 &limit=10…. tới Backend.
4. SensorDataController nhận request và kích hoạt hàm getAllSensorData().
5. Hàm getAllSensorData() xử lý tham số và gọi đến hàm findAll() của SensorDataModel.
6. Hàm findAll() của SensorDataModel được kích hoạt
7. Hàm findAll() của SensorDataModel thực hiện kết nối với Database và yêu cầu lấy dữ liệu.
8. Database thực hiện truy vấn dữ liệu.
9. Database trả dữ liệu cho SensorDataModel

10. SensorDataModel trả kết quả về cho SensorDataController.
11. SensorDataController đóng gói dữ liệu và trả kết quả về cho SensorDataPage.
12. SensorDataPage hiển thị bảng dữ liệu và thanh phân trang cho User.
13. User nhập từ khóa vào thanh tìm kiếm, chọn tiêu chí tìm kiếm hoặc chọn bộ lọc trên giao diện.
14. SensorDataPage kích hoạt lại hàm fetchData() với các tham số mới.
15. Hàm fetchData() kích hoạt và gửi request API GET /api/sensor-datas?page=…&limit=…&… tới Backend.
16. SensorDataController nhận request và kích hoạt hàm getAllSensorData().
17. Hàm getAllSensorData() xử lý tham số và gọi đến hàm findAll() của SensorDataModel.
18. Hàm findAll() của SensorDataModel được kích hoạt
19. Hàm findAll() của SensorDataModel thực hiện kết nối với Database và yêu cầu lấy dữ liệu.
20. Database thực hiện truy vấn dữ liệu.
21. Database trả dữ liệu cho SensorDataModel
22. SensorDataModel trả kết quả lọc về cho SensorDataController.
23. SensorDataController đóng gói dữ liệu và trả kết quả về cho SensorDataPage.
24. SensorDataPage cập nhật lại bảng dữ liệu để hiển thị kết quả tìm kiếm cho User.

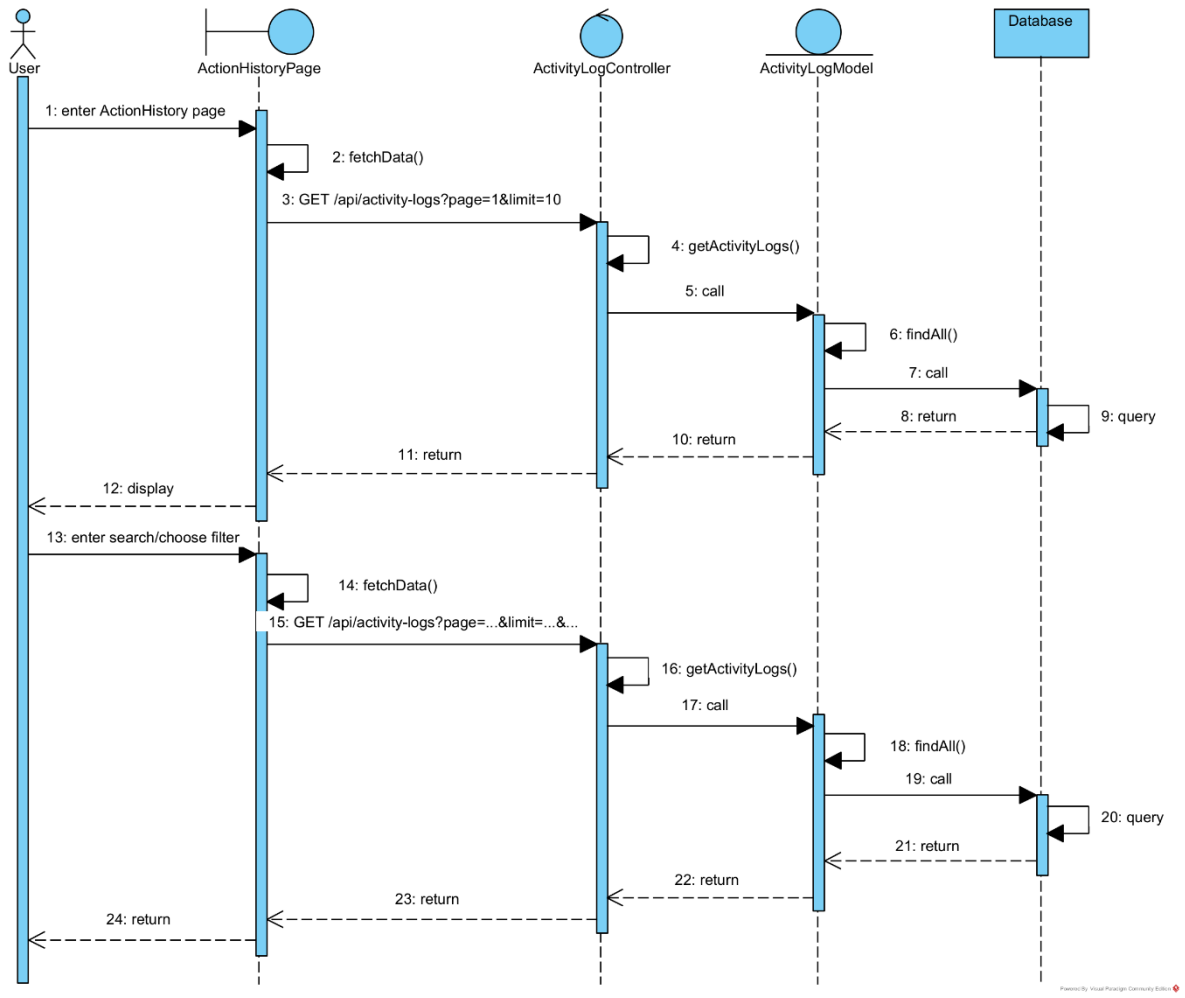


Hình 12. Luồng xem lịch sử dữ liệu cảm biến

3.7.5. Luồng xem lịch sử hành động

1. User truy cập vào trang Action History.
2. ActionHistoryPage khởi tạo giao diện và gọi hàm fetchData().
3. Hàm fetchData() kích hoạt và gửi request API GET /api/activity-logs?page=1&limit=10 tới Backend.
4. ActivityLogController nhận request và kích hoạt hàm getActivityLogs().
5. Hàm getActivityLogs() xử lý tham số và gọi đến hàm findAll() của ActivityLogModel.
6. Hàm findAll() của ActivityLogModel được kích hoạt
7. Hàm findAll() của ActivityLogModel thực hiện kết nối với Database và yêu cầu lấy dữ liệu.
8. Database thực hiện truy vấn dữ liệu.

9. Database trả dữ liệu cho ActivityLogModel
10. ActivityLogModel trả kết quả về cho ActivityLogController.
11. ActivityLogController đóng gói dữ liệu và trả kết quả về cho ActionHistoryPage.
12. ActionHistoryPage hiển thị bảng dữ liệu và thanh phân trang cho User.
13. User nhập từ khóa vào thanh tìm kiếm, chọn tiêu chí tìm kiếm hoặc chọn bộ lọc trên giao diện.
14. ActionHistoryPage kích hoạt lại hàm fetchData().
15. Hàm fetchData() kích hoạt và gửi request API GET /api/activity-logs?page=…&limit=…&… tới Backend.
16. ActivityLogController nhận request và kích hoạt hàm getActivityLogs().
17. Hàm getActivityLogs() xử lý tham số và gọi đến hàm findAll() của ActivityLogModel.
18. Hàm findAll() của ActivityLogModel được kích hoạt
19. Hàm findAll() của ActivityLogModel thực hiện kết nối với Database và yêu cầu lấy dữ liệu.
20. Database thực hiện truy vấn tìm kiếm theo điều kiện lọc.
21. Database trả dữ liệu về cho ActivityLogModel
22. ActivityLogModel trả kết quả lọc về cho ActivityLogController.
23. ActivityLogController trả kết quả về cho ActionHistoryPage.
24. ActionHistoryPage cập nhật lại giao diện hiển thị danh sách nhật ký mới cho User.

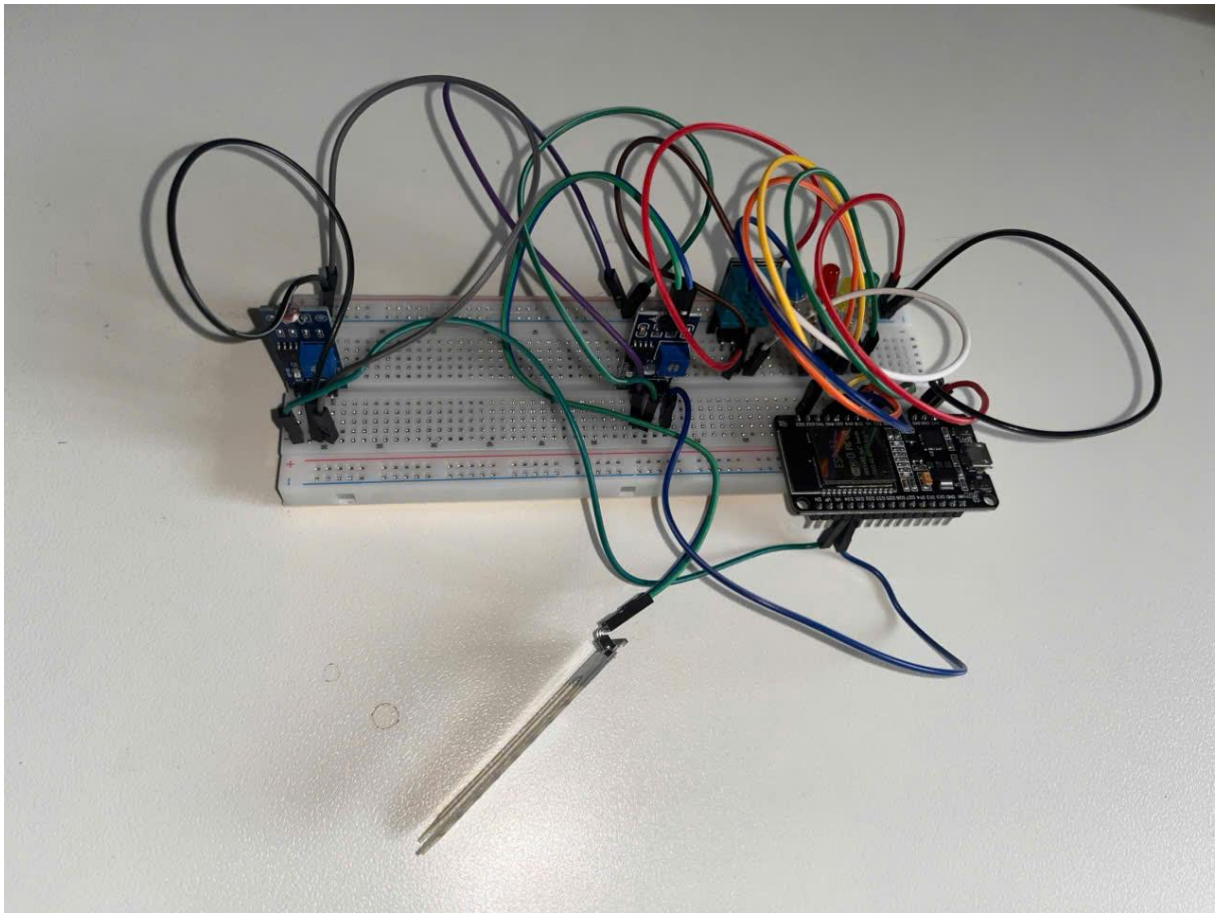


Hình 13. Luồng xem lịch sử hành động

CHƯƠNG 4: XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG

4.1. Triển khai phần cứng

4.1.1. Triển khai sơ đồ phần cứng



Hình 14. Sơ đồ đấu nối phần cứng thực tế

4.1.2. Lập trình phần cứng:

- **Cấu hình thư viện và khởi tạo hệ thống:** Bao gồm việc khai báo các thư viện cần thiết như PubSubClient và ArduinoJson, đồng thời thiết lập các thông số ban đầu cho hệ thống như chân GPIO, Wi-Fi, MQTT Broker và Topic.

```
#include <PubSubClient.h>
#include <WiFi.h>
#include <ArduinoJson.h>
#include <DHT.h>
#include <WiFiClientSecure.h>
// ================= CẤU HÌNH CHÂN LED (THIẾT BỊ MÔ PHỎNG) =================
#define LED_GREEN 17  // pump
#define LED_YELLOW 18 // light
#define LED_RED 19    // fan
#define LED_BLUE 21   // sprayer

// ================= CẤU HÌNH CHÂN CẢM BIẾN =================
#define DHT_TYPE DHT11
#define DHT_PIN 22
DHT dht(DHT_PIN, DHT_TYPE);

#define LDR_PIN 34
#define SOIL_PIN 35

// ================= CẤU HÌNH WIFI & MQTT =================
const char *ssid = "wifi-name";
const char *password = "wifi-password";
const char *mqtt_server_ip = "mqtt-broker-ip";
const int mqtt_port = 1883;
const char *mqtt_username = "mqtt-username";
const char *mqtt_password = "mqtt-password";

// ================= TOPIC =================
const char *topic_data_sensor = "garden/sensordata";
const char *topic_device_control = "garden/devicecontrol";
const char *topic_status = "garden/status";
const char *topic_sync = "garden/sync";

// ================= DEVICE TYPE =================
const String PUMP_TYPE = "pump";
const String LIGHT_TYPE = "light";
const String FAN_TYPE = "fan";
const String SPRAYER_TYPE = "sprayer";
```

Hình 15. Code cấu hình thư viện và khởi tạo hệ thống

- **Cơ chế tự động kết nối lại (Auto-Reconnect):** Hàm reconnect() được xây dựng để đảm bảo hệ thống luôn duy trì kết nối. Khi mất kết nối, ESP32 sẽ tự động thử kết nối lại sau mỗi khoảng thời gian nhất định.

```
void reconnect()
{
  while (!client.connected())
  {
    Serial.print("Dang ket noi MQTT...");
    String clientId = "ESP32Client-SmartGarden-" + String(random(0xffff), HEX);
    if (client.connect(clientId.c_str(), mqtt_username, mqtt_password))
    {
      Serial.println("Da ket noi MQTT Broker thanh cong!");
      client.subscribe(topic_device_control);
      client.publish("garden/sync", "{\"status\": \"connected\"}");
      Serial.println("Đã gửi yêu cầu đồng bộ trạng thái đến Backend.");
    }
    else
    {
      Serial.print("That bai, ma loi=");
      Serial.print(client.state());
      Serial.println(" Thu lai sau 2 giay");
      delay(2000);
    }
  }
}
```

Hình 16. Hàm reconnect() để tự động kết nối

- **Xử lý dữ liệu nhận từ MQTT (callback):** Hàm callback() chịu trách nhiệm xử lý các message nhận được từ Broker, từ đó điều khiển thiết bị như bật/tắt LED theo lệnh từ Backend.

```
// Function process sub message
void callback(char *topic, byte *payload, unsigned int length)
{
  String messageTemp;
  for (int i = 0; i < length; i++)
  {
    messageTemp += (char)payload[i];
  }
  Serial.print("Co tin nhan tu Topic: ["");
  Serial.print(topic);
  Serial.println("]");
  Serial.print("Noi dung: ");
  Serial.println(messageTemp);

  StaticJsonDocument<256> doc;
  DeserializationError error = deserializeJson(doc, messageTemp);

  if (error)
  {
    Serial.print("Loi parse JSON: ");
    Serial.println(error.c_str());
    return;
  }

  String device_type = doc["device_type"].as<String>();
  String action = doc["action"].as<String>();

  int ledState = (action == "ON") ? HIGH : LOW;

  if (device_type == PUMP_TYPE)
  {
    digitalWrite(LED_GREEN, ledState);
    Serial.println(action == "ON" ? "Đã BẬT LED XANH LÁ (Máy bơm)" : "Đã TẮT LED XANH LÁ (Máy bơm)");
  }
  else if (device_type == LIGHT_TYPE)
  {
    digitalWrite(LED_YELLOW, ledState);
    Serial.println(action == "ON" ? "Đã BẬT LED VÀNG (Đèn)" : "Đã TẮT LED VÀNG (Đèn)");
  }
  else if (device_type == FAN_TYPE)
  {
    digitalWrite(LED_RED, ledState);
    Serial.println(action == "ON" ? "Đã BẬT LED ĐỎ (Quạt)" : "Đã TẮT LED ĐỎ (Quạt)");
  }
  else if (device_type == SPRAYER_TYPE)
  {
    digitalWrite(LED_BLUE, ledState);
    Serial.println(action == "ON" ? "Đã BẬT LED XANH DƯƠNG (Phun sương)" : "Đã TẮT LED XANH DƯƠNG (Phun sương)");
  }
  else
  {
    Serial.println("Device ID không khớp với hệ thống!");
    return;
  }

  // --- GỬI PHẢN HỒI (STATUS) VỀ LẠI BACKEND ---
  StaticJsonDocument<256> responseDoc;
  responseDoc["device_type"] = device_type;
  responseDoc["action"] = action;
  responseDoc["status"] = "SUCCESS";
  char responseBuffer[256];
  serializeJson(responseDoc, responseBuffer);

  client.publish(topic_status, responseBuffer);
}
```

Hình 17. Hàm callback() xử lý message nhận từ Backend

- **Đọc, đóng gói và gửi dữ liệu từ cảm biến lên MQTT Broker**: Thực hiện việc lấy dữ liệu từ các cảm biến như nhiệt độ, độ ẩm, ánh sáng và độ ẩm đất thông qua các chân kết nối trên ESP32. Sau đó, sử dụng thư viện ArduinoJson để chuyển đổi dữ liệu cảm biến sang định dạng JSON trước khi gửi đi. Dữ liệu sau khi được đóng gói sẽ được gửi lên MQTT Broker theo các topic đã định nghĩa để backend xử lý và hiển thị.

```
unsigned long now = millis();

  // ================= ĐỌC VÀ GỬI DỮ LIỆU CẢM BIẾN (PUBLISH) =================
  if (millis() - lastMsg > 2000)
  {
    lastMsg = now;

    // 1. Đọc dữ liệu từ DHT11
    float h = dht.readHumidity();
    float t = dht.readTemperature();

    // 2. Đọc dữ liệu Analog (Độ ẩm đất và Ánh sáng)
    int ldr_raw = analogRead(LDR_PIN);
    float light_percent = (1.0 - (ldr_raw / 4095.0)) * 100.0;
    int soil_raw = analogRead(SOIL_PIN);
    float soil_percent = (1.0 - (soil_raw / 4095.0)) * 100.0;

    // 3. Đóng gói dữ liệu thành JSON
    StaticJsonDocument<256> doc;
    doc["temp"] = t;
    doc["humid"] = h;
    doc["soil"] = soil_percent;
    doc["light"] = light_percent;

    char outBuffer[256];
    serializeJson(doc, outBuffer);

    // 4. Gửi lên MQTT Broker
    Serial.print("Gui du lieu: ");
    Serial.println(outBuffer);
    client.publish(topic_data_sensor, outBuffer);
  }
```

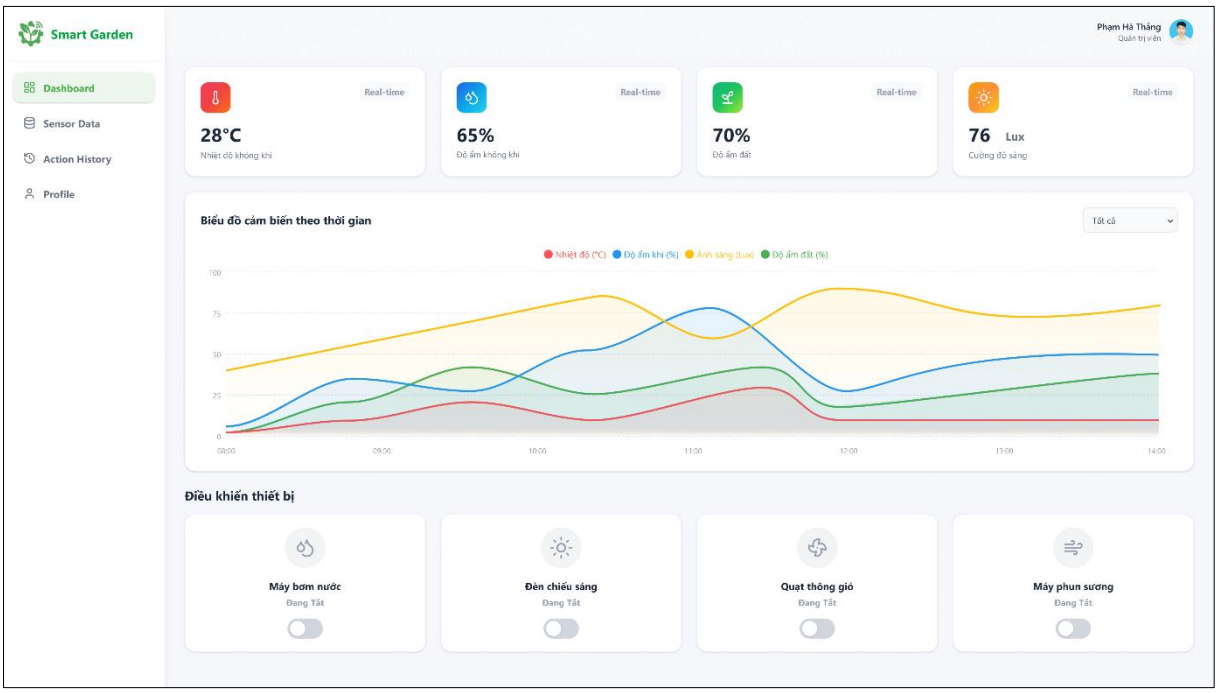
Hình 18. Code đọc, đóng gói và gửi dữ liệu cảm biến cho Backend

4.2. Giao diện người dùng

4.2.1. Giao diện Dashboard

Giao diện Dashboard đóng vai trò là trung tâm quản lý và giám sát của hệ thống, cung cấp cái nhìn tổng thể và trực quan về môi trường vườn thông minh. Giao diện này bao gồm các thành phần chính sau:

- **Thông số cảm biến:** Hiển thị các dữ liệu cảm biến như nhiệt độ, độ ẩm đất, độ ẩm không khí và cường độ ánh sáng dưới dạng các thẻ trực quan, kèm theo tên thông số và đơn vị đo lường tiêu chuẩn.
- **Biểu đồ thời gian thực:** Các biểu đồ động thể hiện xu hướng biến đổi của dữ liệu cảm biến trong khoảng thời gian ngắn, giúp người dùng dễ dàng theo dõi, so sánh và đánh giá sự thay đổi của các yếu tố môi trường.
- **Điều khiển thiết bị:** Cho phép người dùng quản lý và điều khiển trực tiếp các thiết bị trong hệ thống như máy bơm nước, đèn chiếu sáng, quạt thông gió và máy phun sương. Mỗi thiết bị được hiển thị dưới dạng thẻ trực quan với biểu tượng nhận diện rõ ràng, kèm theo công tắc bật/tắt để điều khiển từ xa.

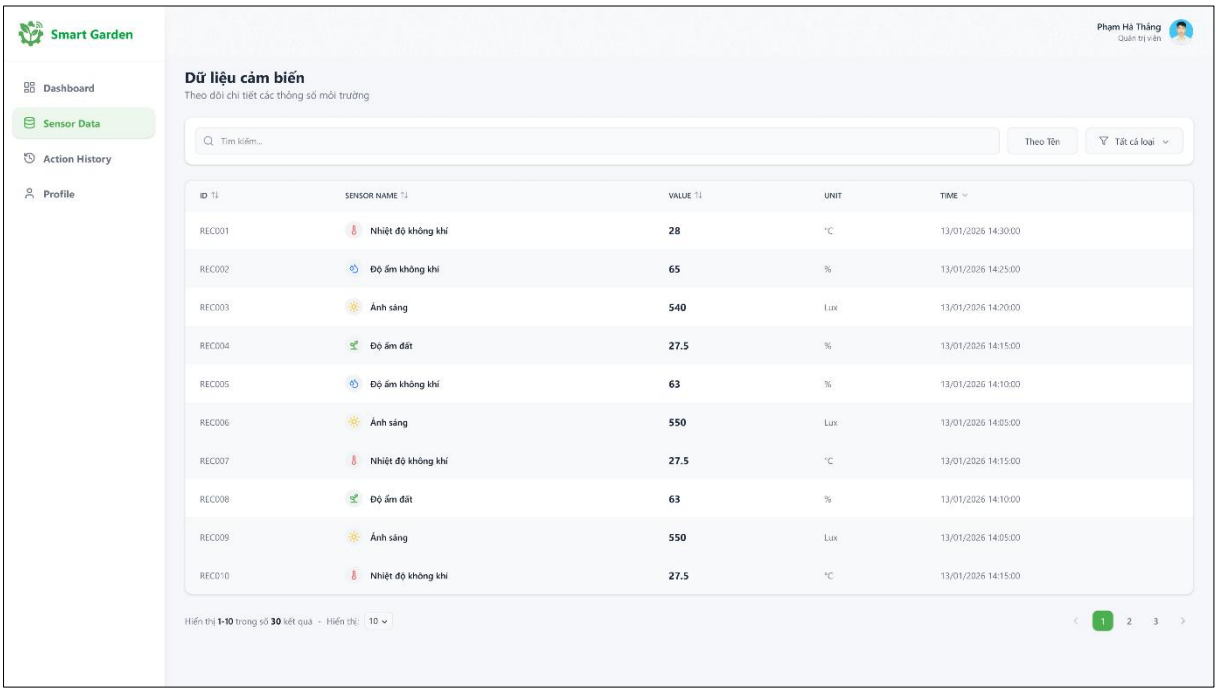


Hình 19. Giao diện Dashboard

4.2.2. Giao diện Sensor Data

Giao diện Sensor Data cho phép người dùng xem, tra cứu và quản lý lịch sử dữ liệu cảm biến đã được thu thập. Giao diện này bao gồm các thành phần chính sau:

- **Thanh công cụ:** Được bố trí ở phía trên cùng, bao gồm ô tìm kiếm cho phép người dùng nhập từ khóa tra cứu, cùng với các nút chức năng hỗ trợ lọc dữ liệu và thực hiện thao tác tìm kiếm.
- **Bảng dữ liệu cảm biến:** Là thành phần trung tâm của giao diện, hiển thị dữ liệu cảm biến được lưu trữ một cách có hệ thống. Các cột trong bảng bao gồm id, tên cảm biến, giá trị cảm biến, đơn vị và mốc thời gian khi dữ liệu được ghi nhận.
- **Phân trang và thông tin hiển thị:** Nằm ở cuối bảng, khu vực này cung cấp thông tin về số lượng bản ghi đang được hiển thị, đồng thời hỗ trợ các nút điều hướng trang, giúp người dùng dễ dàng tra cứu dữ liệu.



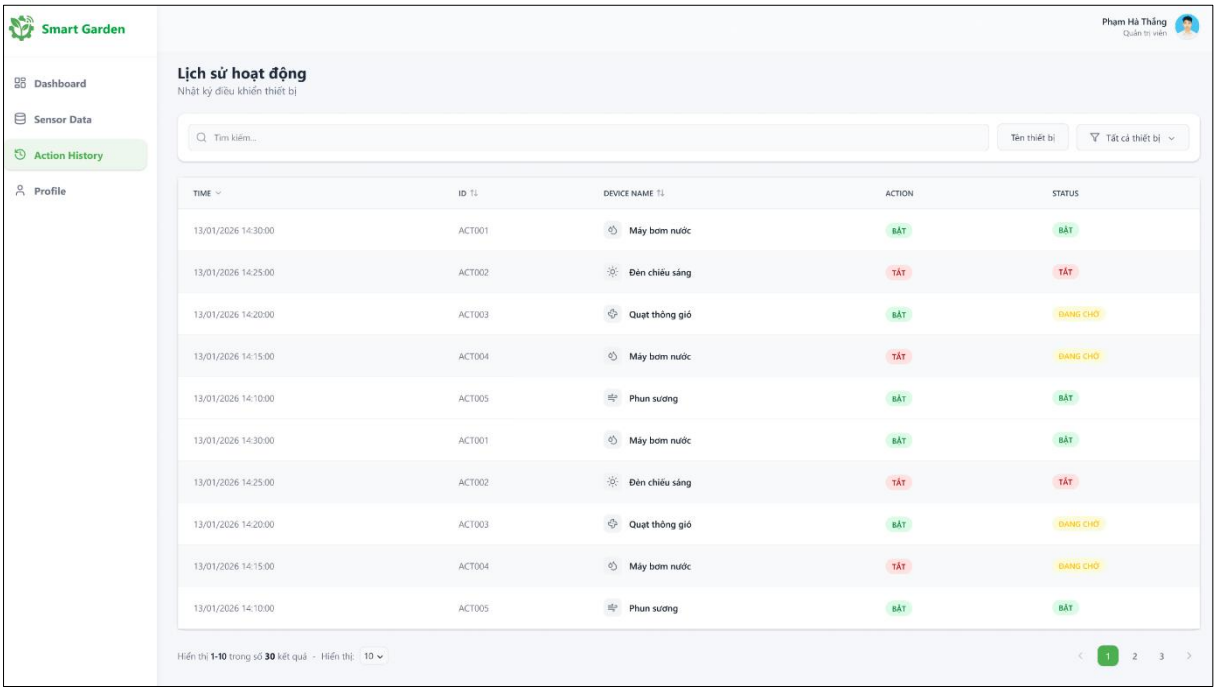
Hình 20. Giao diện Sensor Data

4.2.3. Giao diện Action History

Giao diện Action History cho phép người dùng xem, tra cứu và quản lý lịch sử tất cả các hành động điều khiển thiết bị đã được thực hiện trong hệ thống. Giao diện này bao gồm các thành phần chính sau:

- **Thanh công cụ:** Được bố trí ở phía trên cùng, bao gồm ô tìm kiếm cho phép người dùng nhập từ khóa tra cứu, cùng với các nút chức năng hỗ trợ lọc dữ liệu và thực hiện thao tác tìm kiếm.

- **Bảng dữ liệu hành động:** Hiển thị dữ liệu hành động được lưu trữ một cách có hệ thống. Các cột trong bảng bao gồm thời gian ghi nhận hành động, id của thiết bị, tên thiết bị, hành động và trạng thái
- **Phân trang và thông tin hiển thị:** Nằm ở cuối bảng, khu vực này cung cấp thông tin về số lượng bản ghi đang được hiển thị, đồng thời hỗ trợ các nút điều hướng trang, giúp người dùng dễ dàng tra cứu dữ liệu.

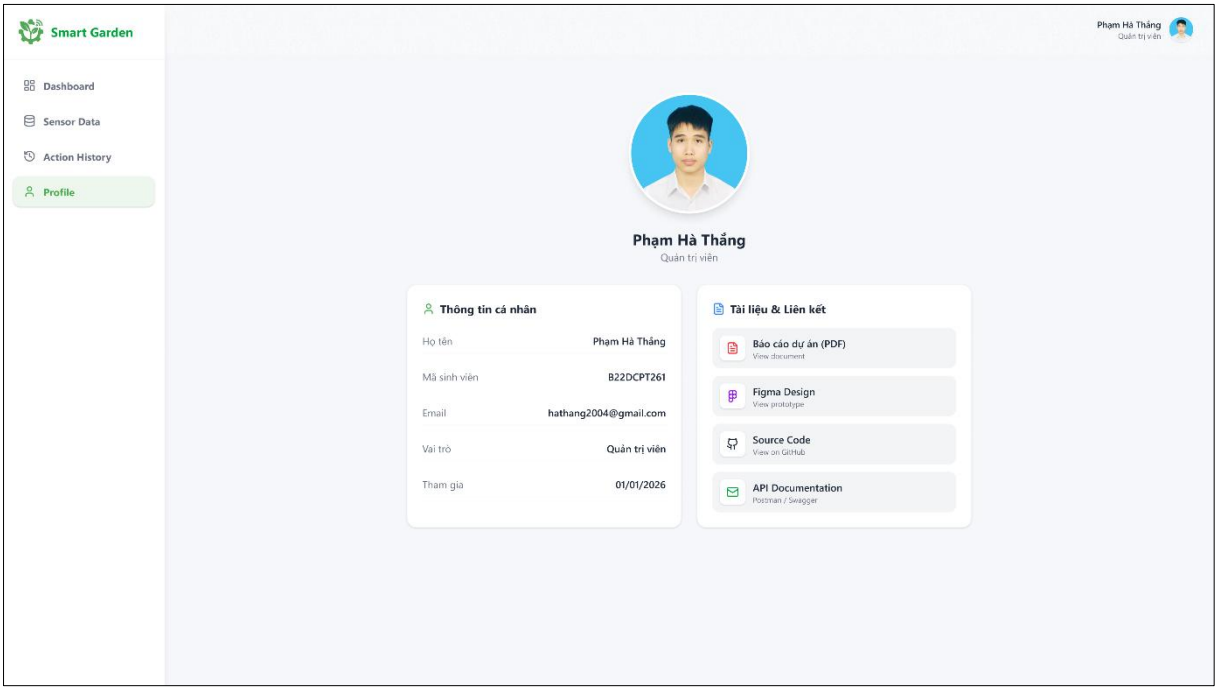


Hình 21. Giao diện Action History

4.2.4. Giao diện Profile

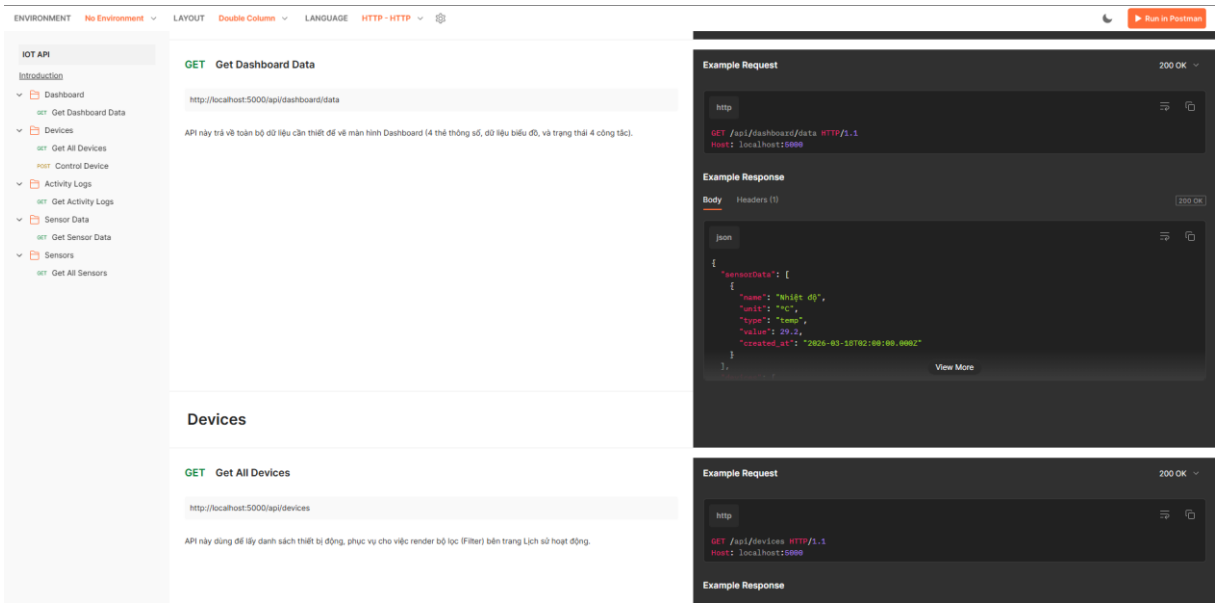
Giao diện Profile cho phép người dùng xem và quản lý thông tin cá nhân của mình trong hệ thống. Giao diện này bao gồm các thành phần chính sau:

- **Avatar người dùng:** Hiển thị hình đại diện của người dùng.
- **Thông tin định danh:** Cung cấp các thông tin cá nhân cơ bản của sinh viên như họ tên, mã số sinh viên và các thông tin liên quan khác phục vụ cho việc xác định danh tính người dùng.
- **Thông tin chi tiết:** Bao gồm các liên kết đến báo cáo dưới dạng PDF, mã nguồn GitHub, tài liệu API và thiết kế giao diện trên Figma, giúp người dùng và giảng viên dễ dàng truy cập.

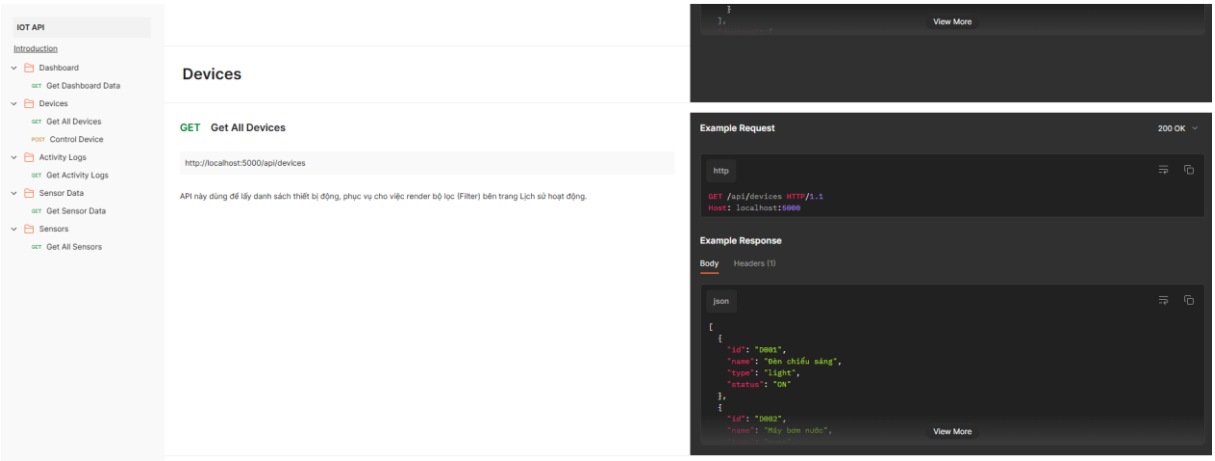


Hình 22. Giao diện Profile

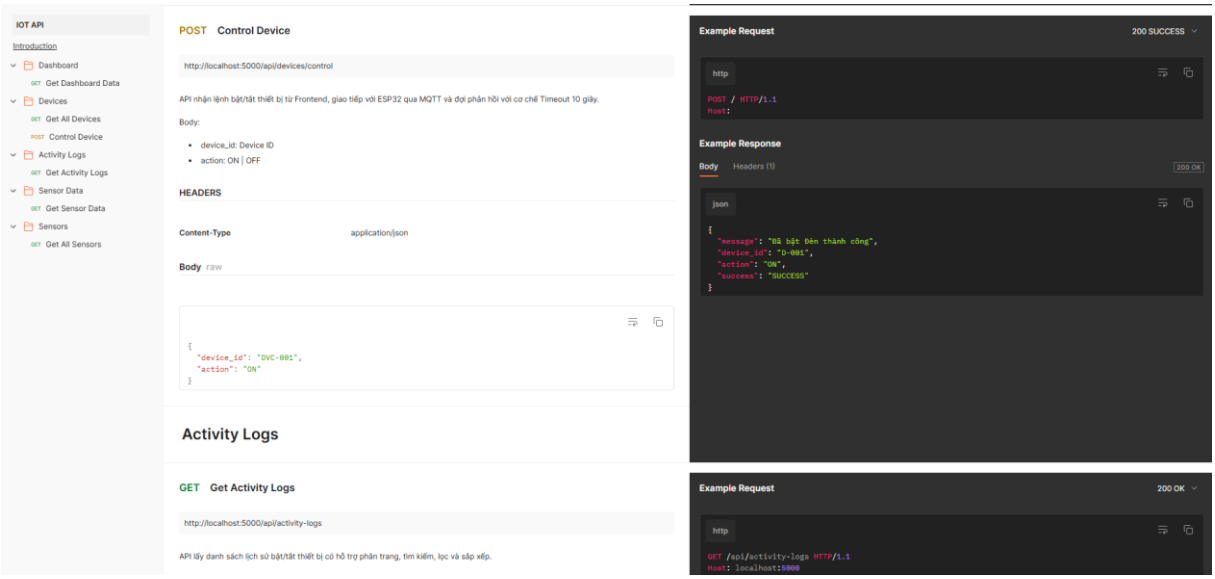
4.3. API Documentation (Postman)



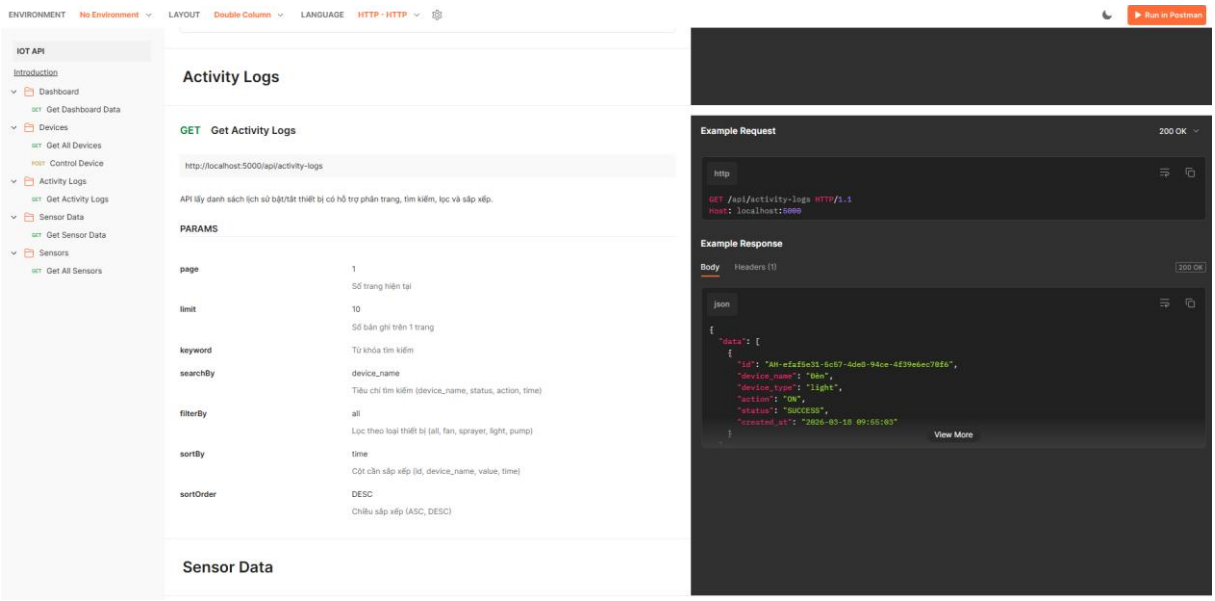
Hình 23. API Documentation (1)



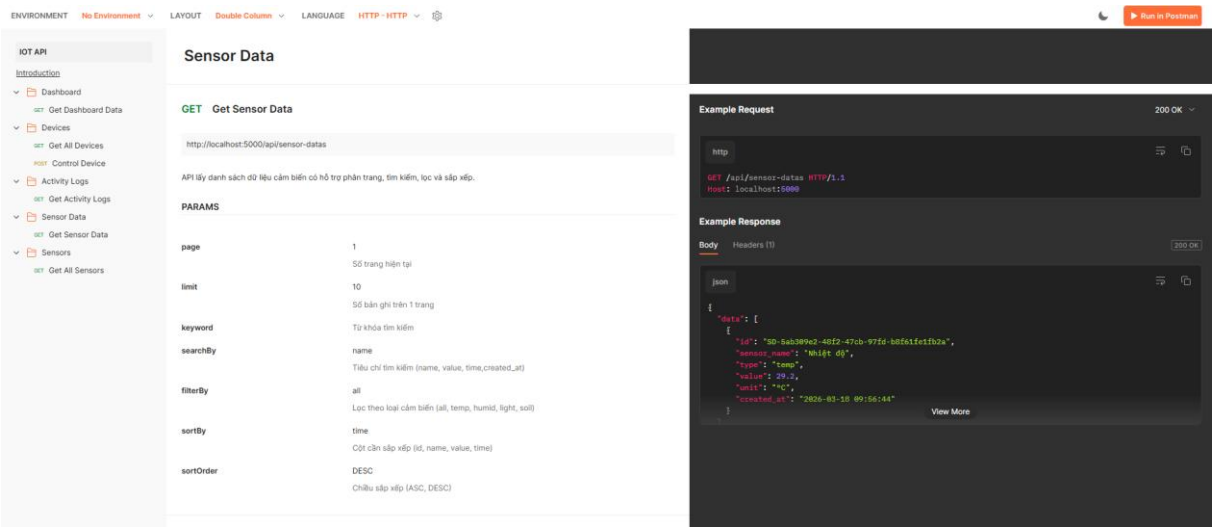
Hình 24. API Documentation (2)



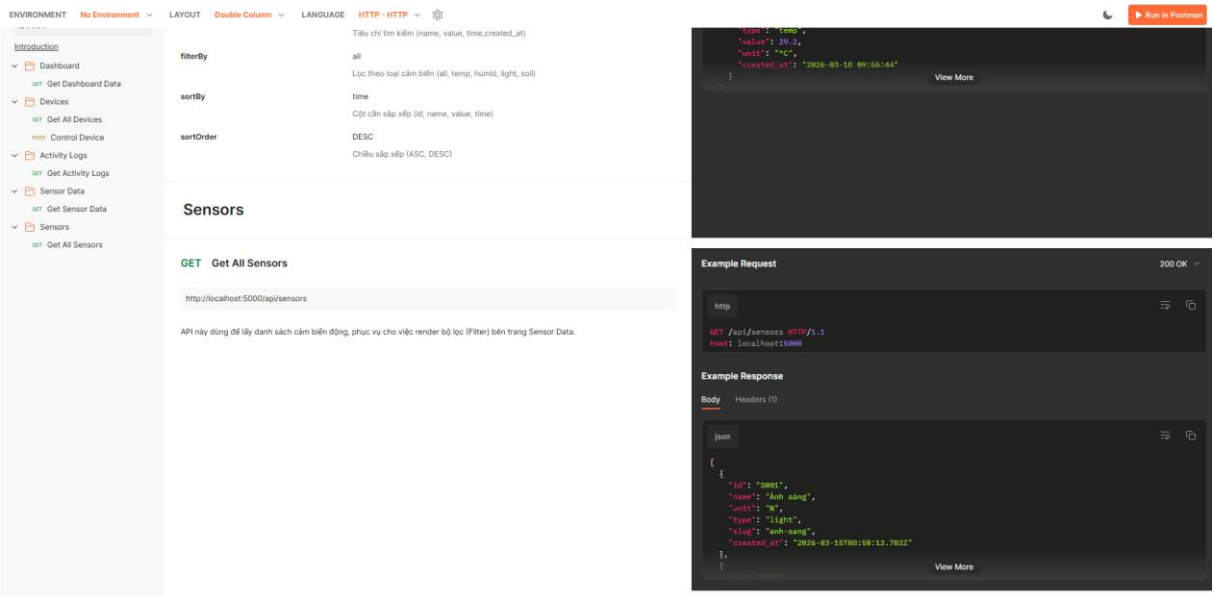
Hình 25. API Documentation (3)



Hình 26. API Documentation (4)



Hình 27. API Documentation (5)



Hình 28. API Documentation (6)

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Sau thời gian nghiên cứu và thực hiện, đề tài *"Xây dựng hệ thống quản lý thiết bị IoT cho vườn thông minh"* đã hoàn thành các mục tiêu đề ra ban đầu:

- Nghiên cứu và áp dụng thành công kiến trúc phần mềm PERN Stack (PostgreSQL, Express, React, Node.js) kết hợp giao thức MQTT để xây dựng một hệ thống IoT hoàn chỉnh từ Phần cứng đến Backend Server và Frontend.
- Hệ thống hoạt động ổn định, dữ liệu cảm biến được cập nhật liên tục với độ trễ thấp.
- Chức năng điều khiển thiết bị từ xa hoạt động chính xác, đảm bảo tính đồng bộ trạng thái và có cơ chế xử lý lỗi (Timeout) chặt chẽ.
- Giao diện người dùng được thiết kế hiện đại, thân thiện, mang lại trải nghiệm tốt nhờ các kỹ thuật tối ưu như Debounce Search và Pagination.

5.2. Hạn chế

Bên cạnh những kết quả đạt được, do giới hạn về thời gian và kinh phí, hệ thống vẫn còn một số điểm cần cải thiện:

- Phần cứng: Hiện tại các thiết bị chấp hành mới chỉ dừng ở mức mô phỏng bằng đèn LED (chưa dùng Rơ-le thật tải điện 220V) và mạch điện vẫn đang cắm trên Testboard, chưa được thiết kế thành mạch in (PCB) công nghiệp để chống chịu thời tiết ngoài trời.
- Bảo mật: Hệ thống chưa tích hợp cơ chế xác thực người dùng (Đăng nhập/Đăng ký). Kênh truyền tải MQTT chưa được mã hóa SSL/TLS, có thể dẫn đến rủi ro bị đánh cắp bản tin điều khiển.

5.3. Hướng phát triển

Dựa trên nền tảng vững chắc hiện tại, đề tài có thể được nâng cấp và mở rộng theo các hướng sau:

- **Tích hợp Trí tuệ nhân tạo (AI):** Áp dụng các thuật toán học máy (Machine Learning) để dự đoán thời tiết hoặc phân tích dữ liệu đất, từ đó hệ thống có

thể tự động đưa ra quyết định tưới tiêu mà không cần sự can thiệp của con người.

- **Hoàn thiện Phần cứng:** Thay thế đèn LED bằng các module Relay thực tế để điều khiển máy bơm, quạt công suất lớn. Thiết kế mạch in (PCB) chuyên nghiệp và vỏ hộp bảo vệ chống nước cho ESP32 khi hoạt động ngoài trời.
- **Bảo mật và Quản lý người dùng:** Bổ sung tính năng Đăng nhập/Đăng ký, xác thực qua JWT (JSON Web Token) và phân quyền người dùng (Admin/User).
- **Phát triển Ứng dụng di động:** Xây dựng thêm phiên bản Mobile App (sử dụng React Native hoặc Flutter) để người dùng có thể quản lý khu vườn dễ dàng hơn trên điện thoại di động thông minh.